

pracy: A Practical Compiler for Attribute-Based Encryption in Python

Sven Argo^{1,✉}, Marloes Venema^{2,✉}, Adrian Ackermann² and Tim Güneysu^{1,3,✉}

¹ Ruhr-University Bochum, Bochum, Germany, firstname.lastname@rub.de

² University of Wuppertal, Wuppertal, Germany, lastname@uni-wuppertal.de

³ Deutsches Forschungszentrum für künstliche Intelligenz, Bremen, Germany

Abstract. Attribute-based encryption (ABE) is a versatile primitive that has been considered in many applications to enforce access control cryptographically. To actually benefit from ABE in practice, we require implementations of schemes that satisfy all the properties that are needed. Many theoretical advancements have been made to attain such properties, ultimately resulting in powerful abstractions such as pair encodings. To build an ABE scheme, we use a compiler (in the theoretical sense), which transforms a provably secure pair encoding scheme into a provably secure ABE scheme. Although several such compilers have been introduced, they all abstract away many details that are relevant for engineers, which can hinder the implementation of schemes in practice.

To address this problem, we propose **pracy**, which is a tool that automatically implements an ABE scheme from an input pair encoding scheme. To achieve this, we first note that we need to overcome a general issue in any automation efforts – including automated optimization and security analysis – in the field of pairing-based cryptography. In particular, there exist no parsers that properly model the interaction between the predicate and the pair encodings. Therefore, we devise a new formal model and type system, which capture this interaction in a way that is compatible with automated implementation efforts. To illustrate the feasibility of our model and system, we construct **pracy**, which is a (practical) compiler in Python that can implement ABE schemes in multiple target programming languages such as Python and C/C++. With **pracy**, we not only make the implementation of ABE schemes from pair encodings more accessible to practitioners, we realize the potential that pair encodings have to simplify implementation efforts.

Keywords: attribute-based encryption, implementation, compiler, automation

1 Introduction

Attribute-based encryption is a powerful type of public-key encryption that enforces attribute-based and role-based access control on a cryptographic level [SW05, GPSW06a, BSW07], and has therefore been considered for various applications [KL10, SRGS12, WMZV16, ETS18, VAH23, LVV⁺23]. For the past two decades, the field of pairing-based ABE has matured substantially, resulting in schemes satisfying desirable properties such as the support of large universes [Wat11, LW11b, RW13, RW22], multiple authorities [Cha07, LW11a, RW15, Ven23b, RVV24] and negations [OT10, Att19, TKN20, AT20, Ven23b, RVV24]. Furthermore, several implementations of ABE exist [AGR12, FEN, dIPVA22a, LVV⁺23], highlighting both their practical feasibility and efficiency.

However, over the past years, theoretical and practical efforts have somewhat diverged. The most powerful schemes have followed from *pair encodings* [Att14, Wee14, CGW15,

AC16, AC17, ABGW17, RW22, Ven23b, RVV24], which is an abstraction of many pairing-based ABE schemes that considers the exponent space. In this paradigm, pairing-based ABE schemes are constructed via a *compiler*, which takes as input a pair encoding scheme (PES) and potentially some other information, and then transforms the PES into an ABE scheme. The advantage of this paradigm is that it simplifies the design and analysis of schemes, because we need to prove simpler security properties such as the symbolic property [AC17, Ven23b, RVV24], which are purely algebraic. The disadvantage is that pair encodings abstract away many details that are relevant for practitioners, and therefore, they impede implementation efforts. That is, implementation is no longer a translation from a mathematical description of the scheme, but requires a translation from PES to ABE first, which can be an error-prone and time-consuming task to do manually.

To solve this issue, it is a natural question to ask: can we automate some efforts around translating PES to ABE? To answer this question, we consider various works on automation in pairing-based ABE, centered around optimization [AGH13, AGOT14, AGH15, AHO16] and security analysis [ABGW17, dIPVA23, RVV24]. Although these works are powerful in their own right, there is an important building block that is missing, which we require to make substantial improvements to the automation of this translation for PES. In particular, none of these automated tools has a parser that can handle the predicates in the scheme, e.g., the attribute sets associated with the keys and the policies associated with the ciphertexts (in ciphertext-policy ABE [BSW07]). Instead, all of those tools can only be run on schemes instantiated with fixed, e.g., sets and policies. Because the integration of the predicates is an important aspect in how schemes are implemented, this makes the effect that such parsers could have on the automation of translating PES very limited.

In this work, we aim to bridge the gap between theory and practice by introducing **pracy**, which is a tool that automatically translates a PES into an *implementation* of an ABE scheme. To achieve this, we devise the necessary theoretical foundations that model the interaction between pair encodings, their associated predicates and the compiled ABE scheme. Specifically, we design a formal model and type system that capture many of the aforementioned schemes, and that are compatible with what we require for automation. Building on these foundations, **pracy** automatically implements an ABE scheme based on the input PES in any supported language, e.g., C/C++ or Python. In this way, we not only make theoretical compilers more accessible to practitioners, we also leverage its potential to simplify the implementation of schemes. Implementing schemes becomes as simple as creating a JSON file – largely using $\text{\LaTeX}$ syntax – essentially containing just the description of the PES. We therefore benefit from pair encodings not only in the theoretical setting, but also in practice.

1.1 Our Contributions

This work encompasses two major and four minor contributions.

- **A bridge from theory to practice: formal model and type system.** Although pair encodings and ABE compilers have proven to be a valuable abstraction for theoreticians, practitioners benefit little from them and often need to manually concretize specifications for their implementations. In a sense, compilers could even harm practical efforts because they abstract away important steps in the implementation. In this work, we provide the theoretical foundations that are needed to automatically transform PES into ABE. These theoretical foundations consist of a formal model and type system that model the interaction between pair encodings, the predicates and the compiled ABE scheme. This formal model and type system are aimed to be applicable to other forms of automation in pairing-based ABE as well, e.g., automated optimization and security analysis.
- **A new tool: pracy.** We develop **pracy**, the first ever **practical** attribute-based

encryption (ABE) compiler written in Python. The tool is designed with modularity, extensibility and usability in mind to become a solid foundation for future research. The intuitive input format is based on JSON and $\text{\LaTeX}$ to blend easy processing with conventional notation.

- **Various secondary contributions.** We also provide several secondary contributions, which are either a direct result of `pracy`, or are part of our aim to make the tool usable. These can be summarized as follows:
 - **New implementations.** Via our tool, we obtain functional implementations of schemes from literature which, to our knowledge, have never been implemented.
 - **Discovery of notational “bugs” in existing work.** Using our tool, we discover various notations used in literature that could be dangerous when implemented in practice.
 - **More natural representation of decryption in pair encodings.** To make the JSON format more user friendly, we introduce a more natural yet equivalent representation of the decryption algorithm in the pair encodings definition.
 - **More efficient access structure for PESs.** We uncover a subtle issue with how access structures are defined in many PESs, making it less applicable. We improve the representation for such PESs and show that this change in the ABE scheme is provably secure if the scheme from the original PES is secure.

1.2 Related Work

Several works on PES-based compilers have considered the implementation of schemes and automation of complex tasks in the field of pairing-based cryptography.

Theoretical Compilers for Pair Encodings. Besides the Ven23 and ISABELLA compilers, there are also many other compilers using pair encodings [Att14, Att16, AC16, AC17, ABGW17, RW22] or predicate encodings [Wee14, CGW15] (which is a subclass of pair encodings [ABS17]). The main differences between those and the Ven23 and ISABELLA compilers is in the security models that are used, as well as the coverage of the compilers.

On the one hand, most of the other compilers can achieve fully adaptive security in the standard model, except [ABGW17, RW22], which both achieve fully adaptive security in the generic group model. In contrast, ISABELLA achieves both static security under a q -type assumption, and fully adaptive security in the generic group model. Although the generic group model is an idealized model, in practice, schemes proven secure in it have not been shown to be weaker than schemes not requiring the generic group model [RW22].

On the other hand, the coverage of Ven23 and ISABELLA is much larger, i.e., the class of encodings that can be covered is broader, e.g., it includes (decentralized) multi-authority schemes [Cha07, LW11a], which cannot be covered with the other compilers. Furthermore, they can support other desirable features such as efficient instantiations in the pairing groups, and the use of hash functions that map strings into the pairing groups (also necessary for multi-authority schemes).

Simplified Implementation of Attribute-Based Encryption. Because implementing pairing-based schemes is difficult, various tools have been proposed to simplify this effort. The most notable is Charm [AGR12], which allows for rapid prototyping of pairing-based schemes in Python. To facilitate this, it contains implementations of many important building blocks needed to implement ABE, e.g., policy and set parsers, access structures with associated secret sharing schemes, and group and pairing operations. Although this simplifies the implementation of prototypes considerably, Charm relies on outdated pairing

groups, making it infeasible to use any of its implementations in practice [dlPVA22a]. Furthermore, it cannot automatically transform pair encoding schemes into full-fledged implementations of ABE schemes, simply because it was not designed to do so.

In contrast, Ambrona, Barthe and Schmidt [ABS17] have designed a library called **abe-relic** (as part of their paper on general transformations on pair and predicate encodings) that simplifies the implementation schemes constructed from predicate encodings, built on top of the RELIC toolkit [AGM⁺]. Although it is closer to the goal of automatically implementing ABE schemes from input pair or predicate encoding schemes, it does not fully automate this and still requires much manual effort. Additionally, in contrast to Charm, which has been used extensively by the community and is still maintained, **abe-relic** has not been updated in nine years.

Automated Tools for Pairing-Based Cryptography. Due to the complexity of designing secure pairing-based schemes as well as optimizing them (on a structural level), many automated tools have been introduced. For example, various works on optimization problems have focused on automatic batching (of signatures) [AGHP12], group assignment [AGH13, AGOT14, AGH15, AHO16], or pairing-product equation testers [HV19].

Other works have focused on automated security analysis and proofs [BFF⁺14, BGS15, ABS16, ABGW17, dlPVA23, RVV24]. Of the latter tools, only [ABGW17, dlPVA23, RVV24] cover ABE for more expressive policies such as Boolean formulas, but these still require the user to analyze schemes for sets and policies that are fixed upon analysis. Although these tools make much progress in the automation of complex tasks such as security analysis, there remains an important gap to be bridged. That is, none of these tools contains a parser for PESs that can also parse the associated policies and attribute sets. Having such a parser could improve these works, making them more generally applicable.

1.3 Technical Details

In this work, we aim to unify existing work on automated implementation and simplified design of ABE schemes by introducing **pracy**. In particular, we provide both the theoretical foundations for automated implementation of ABE schemes, and implement a tool called **pracy** that builds on these foundations. The main hurdle that we overcome is that, currently, none of the parsers provided by any of the aforementioned works properly captures the interaction among all relevant components. To achieve this, we design a customized type system that properly models pair encoding schemes, as well as their relationship with the predicate (consisting of, e.g., policies and attribute sets). We show below that there are many intricacies that need to be captured by such a system to make it suitable as a foundation for automated implementation.

1.3.1 Background

Pairing-Based (Decentralized) Ciphertext-Policy ABE and Predicates. We focus on *ciphertext-policy ABE* (CP-ABE) [BSW07], as it has been shown to be interesting for many practical applications [SRGS12, VAH23, LVV⁺23]. In CP-ABE, the encrypting user can specify a policy that describes who is allowed access to the encrypted data. In turn, a decrypting user, who has a secret key for an attribute set that satisfies the policy, can decrypt the ciphertext and access the data. This secret key is derived from a master public and secret key, generated by an authority that is assumed to be trusted. To reduce the amount of trust, the role of the authority can also be shared among multiple authorities, which is called *multi-authority ABE* [Cha07, LW11a].

A more general definition of ABE covers *predicates*. A predicate $P: \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ takes as input a ciphertext predicate $x \in \mathcal{X}$ and a key predicate $y \in \mathcal{Y}$ and evaluates to 1 (“true”) if the key predicate satisfies the ciphertext predicate. The idea is that the

ciphertext for x can be decrypted by a secret key for y if $P(x, y) = 1$, and that the message remains computationally hidden if not.

We build schemes from *pairings*, which are a map $e: \mathbb{G} \times \mathbb{H} \rightarrow \mathbb{G}_T$ over three groups $\mathbb{G}, \mathbb{H}, \mathbb{G}_T$ of prime order p , with generators $g \in \mathbb{G}$, $h \in \mathbb{H}$, such that for all $x, y \in \mathbb{Z}_p$, we have $e(g^x, h^y) = e(g, h)^{xy}$.

The Ven23 and ISABELLA Compilers. We give a brief overview of the Ven23 [Ven23b] and ISABELLA [RVV24] compilers, which can be used to build pairing-based CP-ABE schemes from pair encodings. At a high level, these compilers construct the master public keys, the user secret keys and ciphertexts of an ABE scheme in the following way:

$$\begin{aligned} \text{MPK} &= (e(g, h)^\alpha, g^{\mathbf{b}}), \quad \text{USK}_y = (h^{\mathbf{r}}, h^{\mathbf{k}(\mathbf{r}, \hat{\mathbf{r}}, \alpha, \mathbf{b}, y)}), \\ \text{CT}_x &= (M \cdot e(g, h)^{c_M}, g^{\mathbf{s}}, g^{\mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}, x)}, e(g, h)^{\mathbf{c}'(\mathbf{s}, \hat{\mathbf{s}}, \alpha, x)}). \end{aligned}$$

In the above, the pair encodings are defined by the master-key variables α , common variables $\mathbf{b} = (b_1, \dots, b_n)$, the key encodings $(\mathbf{r}, \mathbf{k}(\mathbf{r}, \hat{\mathbf{r}}, \alpha, \mathbf{b}, y))$ and the ciphertext encodings $(\mathbf{s}, c_M, \mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}, x), \mathbf{c}'(\mathbf{s}, \hat{\mathbf{s}}, \alpha, x))$, where $\mathbf{b}, \mathbf{r}, \hat{\mathbf{r}}, \mathbf{s}, \hat{\mathbf{s}}, \mathbf{s}$ denote vectors of variables and $\mathbf{k}, c_M, \mathbf{c}$ and $\mathbf{c}'$ denote (vectors of) polynomials over those variables. Also, M is the message.

The compilers also enable advanced features such as flexible group assignment, and large-universe and multi-authority support, by defining two assignment functions on top of the PES. The first is a group assignment function, which specifies in which groups $\mathbb{G}$ or $\mathbb{H}$ each variable and polynomial is instantiated with the compiler. The second is a full-domain hash assignment function, which specifies for certain variables whether they can be generated via a hash function that maps arbitrary-length strings into the pairing group. To ensure that these functions are compatible with the used pairing groups and decryption correctness, the compilers impose correctness properties on those functions.

1.3.2 The Gap: Interactions between the Predicates and Pair Encodings

Although pair encodings and the associated assignment functions are defined unambiguously and are already explicitly typed in a sense, their interaction with the input predicates x and y is mostly implicit and completely abstract. However, modeling this interaction is crucial in the full automation of any efforts around ABE, including but not limited to implementation. That is, to be able to run key generation, encryption and decryption for any input predicate, e.g., an attribute set or policy, any tool would need to model an abstraction of these input predicates. Moreover, for an implementation tool, this abstraction needs to interact with concrete instantiations of those input predicates.

Example. To illustrate the problem, we consider an example. In CP-ABE, the policies are modeled in practice as Boolean formulas. In the scheme, we implement these policies using linear secret sharing matrices (to describe the Boolean formula) and functions that map the rows of those matrices to concrete attributes (to describe the literals in the Boolean formula). Suppose that λ_j is the secret share for the j -th literal in the policy provided using this secret sharing scheme, and let $\rho(j)$ be the associated attribute. Then, one of the ciphertext polynomials for this particular input predicate can be $c_{1,j} = \lambda_j + s_j b_{\rho(j)}$ (see, e.g., [Wat11]). This example shows, among other things, that the indices of the variables and polynomials are dependent on the input predicate. For instance, the ciphertext polynomial $c_{1,j}$ is indexed in a constant and the index of the literal of the input policy, and the common variable $b_{\rho(j)}$ is indexed in the variable that is generated with the function applied to the index of this literal. Furthermore, there is an implicit universal quantifier, because the ciphertext polynomial needs to be computed in this way *for all* literals j , i.e., we compute $\{c_{1,j} = \lambda_j + s_j b_{\rho(j)}\}_{j \in \{1, \dots, n_1\}}$ for all n_1 literals.

1.3.3 Our Formal Model and Type System

To make the interaction between the predicates and pair encodings concrete, we design a formal model and type system that cover a large class of predicates. We do this by examining many ABE schemes and the predicates they cover. By unifying the approaches used to construct predicates, we can construct our foundations in such a way that they will be able to capture many schemes, and that possible extensions (e.g., to cover future schemes or other classes of schemes) can build on our foundations. In general, we observe that all schemes have a similar structure. The indices of the variables are related to the predicates, and that the polynomials (and non-lone variables) are quantified over sets, e.g.,

$$\{c_{1,j} = \lambda_j + s_j b_{\rho(j)}\}_{j \in \{1, \dots, n_1\}}$$

indices universal quantification

Note that some functions may be applied to the elements, which can occur both in the quantifier or the indices, e.g., $b_{\rho(j)}$ or $j \in \rho(\{1, \dots, n_1\})$. To model this, we always write the indices as functions, and use the identity function when no function is applied. This example also shows that the indices and quantification are closely related (potentially via those functions): j in the indices of $c_{1,j}$ and λ_j are from the set $\{1, \dots, n_1\}$ in the quantification. This relationship needs to be modeled as well.

Equivalence. An additional difficulty is that we need to model “equivalence” in a meaningful sense. This is important, because the key and ciphertext components are built from the master public and secret keys, but their indices may differ in syntax. For instance, the master public key matching our example may consist of $\{b_{\text{attr}}\}_{\text{attr} \in \mathcal{U}}$, where $\mathcal{U}$ denotes the universe of all attributes supported by the system. To be able to identify which public key components need to be used in generation of $c_{1,j}$ in the encryption, we need to be able to infer from $b_{\rho(j)}$ that it is equal to a variable in the public key, despite having different syntax in their indices. Regardless, we see that ρ maps the set $\{1, \dots, n_1\}$ to a set of attributes, that is in turn a subset of the universe. Hence, to model this equivalence, we use that the *types* of the elements in these sets are equal.

Type System for the Predicates. Hence, we develop a type system that allows us to model the relationship between the indices and quantifier as well as the equivalence in the aforementioned sense. To do this, we systematically analyze many existing schemes, see e.g., [VAH23] for a comprehensive overview of schemes. We aim to cover both simpler schemes for Boolean formulas without negations [Wat11, RW13], and more complex ones, e.g., schemes supporting negations [TKN20], multiple authorities [RW15, Ven23b] or both [Ven23b]. From these, we extract basic types for elements, e.g., attribute values, labels or authority identifiers, but also set types, e.g., the attribute universe or attribute sets, and functions, e.g., mapping the literals to attributes or authorities.

Type System for the Pair Encodings. Furthermore, our type system needs to cover pair encodings. For most of the variables and polynomials, this is rather straightforward, as they are already typed in the definition of pair encodings, e.g., non-lone key variables or ciphertext polynomials. There are, however, some special types of variables that need to be captured in a certain way in the implementation. For example, $c_{1,j}$ contains a share λ_j , which needs to be implemented following the specifications of the linear secret sharing scheme. Based on our systematic analysis, we also identify several such “special variables” and unify their usage in concrete schemes.

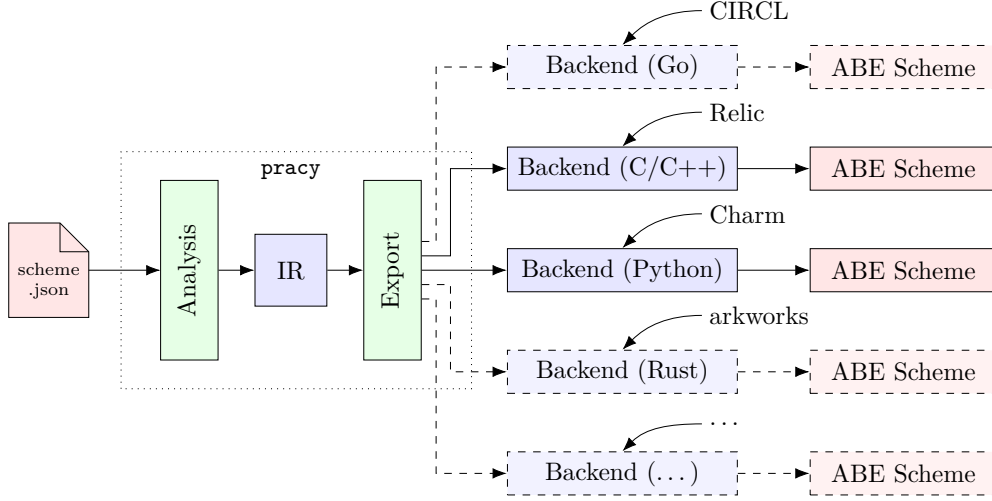


Figure 1: A high level overview of **pracy** and its relation to its backends and existing libraries. The abstract intermediate representation (IR) enables us to support various (future) technologies and frameworks. Solid lines indicate components included in this work whereas dashed arrows and boxes in lighter colors are ideas for future work.

Correctness Assurances. The type system also enables more complex analyses of the input scheme instead of straightforward (naive) translation. As a result, we can detect and prevent many potential ambiguities and inconsistencies, both in the input as well as the generated code. This makes **pracy** more user-friendly and reliable and imposes less restrictions on the features and behavior of the backend technology.

1.3.4 Implementation of **pracy**

To illustrate the effectiveness of our theoretical foundations, we present **pracy**, which is an implementation of a compiler based on our formal model and type system (Figure 1). At a high level, **pracy** is written in Python and takes as input a JSON file that specifies the pair encoding scheme and the two assignment functions. It then parses the file, deriving all the types and relevant functions, internally creating trees that appropriately represent each variable and polynomial as well as their (predicate-related) indices and quantifications. The compiler then performs an extensive analysis on the internal representation of the scheme, which includes various correctness and type checks. After the analysis is completed, the compiler generates an implementation of the theoretically compiled scheme (via Ven23 and ISABELLA) in a customized intermediate representation (IR). This IR-based implementation can then be exported to any target programming language that interacts with this IR, e.g., higher-level languages such as Python or lower-level ones such as C/C++. Lastly, to use the exported code in practice, we can build a backend around it, which implements e.g., the group and pairing operations, policy parsers, secret sharing schemes or benchmarking functionalities. To show the versatility and potential of such backends, we have implemented two functional backends: one in C/C++ using the RELIC toolkit [AGM⁺], and one in Python using the Charm framework [AGR12].

Modularity, Extensibility and Usability. In our implementation of **pracy**, we aim at achieving modularity, extensibility and usability. In the first place, we attain this *modularity* by building on theoretical foundations that strive to be broadly applicable, i.e., the Ven23 and ISABELLA compilers and our formal model and type system. In the second place, the intermediate representation is designed with the support of different types of programming

languages in mind, e.g., dynamically typed (like Python) or statically typed (like C/C++). This is also supposed to make our compiler more future-proof than existing libraries such as Charm, as it will allow us to write exports and backends for programming languages that may turn out more desirable, e.g., Rust or Go.

Furthermore, this kind of *extensibility* is also intrinsic in the architecture of our compiler. For example, we have included placeholder layers for, e.g., structural optimizations and code optimizations. Lastly, we aim at *usability* by ensuring that the JSON format describing the input scheme is user friendly, and in particular is close to the mathematical description of concrete pair encoding schemes, and requires minimal user input. Additionally, the tool provides fine-grained feedback on the input scheme when it is incorrect in some aspects, so that it can help in the design of new schemes. For example, the tool can determine quickly whether the chosen group assignment is compatible with the scheme.

1.3.5 Demonstrating pracy

We also demonstrate the use of `pracy`, and show that implementations automatically generated with our compiler are already close to practical despite the lack of optimizations. To this end, we implement several (configurations of) schemes from literature using our tool, varying from CP-ABE for Boolean formulas without negations [Wat11, RW13, AC17] to decentralized multi-authority CP-ABE schemes with and without negations [Ven23b].

On the one hand, this shows that the JSON descriptions of these schemes are close to their mathematical descriptions in literature, while automatically implementing the compiled schemes takes seconds rather than hours or days. Furthermore, features that are usually time-consuming and error-prone to add are now much easier to support. For example, we can easily flip the groups of the key and ciphertext components to tweak the scheme’s efficiency [dlPVA22a] by changing a single character, or transform the scheme from a small to a large-universe setting by adding one extra line in the JSON description.

On the other hand, our evaluation shows that the “cost” of this automation is limited. Both the code sizes and the running times of the implementations are, in most of the cases, only a couple of factors worse compared to manual(ly optimized) implementations of those schemes. The only exception is the decryption of the AC17 scheme [AC17], which can be optimized in a way [dlPVA22a] that we cannot support with our tool yet. This also illustrates that there is much potential for future improvements using such optimizations.

1.3.6 Secondary Contributions

We also provide several secondary contributions. Some of these are a direct result of our work, whereas others are part of our aim to make the tool more usable.

New Implementations. As a direct result of our compiler and backend implementations, we obtain implementations of two schemes for which previously no implementations existed, i.e., two of the decentralized schemes in [Ven23b] (Section 5.1 and Appendix E.1).

Discovery of Notational “Bugs”. Another direct result of our work is the discovery of potentially dangerous notations that can be used in the formulation of schemes. For example, if the “name” of the variable is the same but the indexing strategy suggests that the variables are not the same, e.g., $\{b_{\text{attr}}, b_{\text{lab}}\}_{\text{attr} \in \mathcal{U}, \text{lab} \in \mathcal{L}}$. In this case, the implementation of a scheme using public keys associated with these variables could confuse the two, which may be exploitable by an attacker. Another example of such a dangerous notation is having collisions in the indices, e.g., for a single predicate, there are two attributes that yield the same index of a key polynomial. When implemented, the key associated with the first attribute may be overwritten by the second.

More Natural Representation of Decryption in Pair Encodings. To make the JSON file more user friendly, we introduce a more natural representation of the decryption in pair encodings that is closer to the mental model and easier to specify in code. Usually, decryption is written in matrix form, so that decryption can be written as a product, e.g., of the vector of non-lone key variables and the vector of ciphertext polynomials. This matrix is typically sparse, i.e., for relatively few key-ciphertext pairs, the associated entry in the matrix is nonzero. Hence, for our representation of decryption, we require only the specification of those pairs for which the associated entry is nonzero.

More Efficient Access Structures for Pair Encodings. To make any handling of the access structures in the backend more modular, we improve the representation of access structures in pair encodings. Currently, all existing PESs captured by, e.g., the ISABELLA compiler require a specific representation of the linear secret sharing scheme. By slightly adapting this representation in the PES, we can make it more modular, making the implementation of the secret-sharing mechanism less implementation-dependent. Because this adaptation is technically not covered by the definition of PES, we cannot use any of the security results on which our work builds. Therefore, we define the notion of approximate pair encodings, which extends pair encodings to cover the adaptations (and potentially other interesting adaptations that can improve concrete schemes). We also prove that the security of any scheme compiled from a approximate pair encoding scheme can be reduced to that of a scheme compiled from the actual pair encoding scheme it approximates.

1.4 Our Work as a Foundation for Other Efforts

We have designed `pracy` in such a way that it can be used as a foundation for multiple other efforts around simplified implementation and automation. Most obviously, we have included placeholders for optimization, which can be used to improve the automatically generated implementations of the schemes, e.g., using automated group assignment tools [AGH13, AGH15]. Furthermore, our theoretical foundations and the parser and analysis layers of our compiler can be useful in improving automation efforts in computer-aided security proofs [ABGW17, dIPVA23, RVV24], whose parsers cannot consider abstractions of predicates and currently can prove security of schemes for fixed instances of predicates only. Ultimately, it would be interesting to integrate such an automated prover in `pracy`, so that we obtain high security guarantees (from a theoretical standpoint) for our implementations.

Another line of research that would be interesting to explore that can use our work as a foundation is the simplified implementation of ABE prototypes as well as their comparison with other schemes. Currently, Charm seems to be the de-facto choice to prototype new schemes and compare them to existing ones, mainly because it is easy to use and contains many tools that are needed to implement ABE and related primitives. Because one of our backends utilizes Charm, we can use `pracy` to simplify the implementation of prototypes in Charm even further. Alternatively, other backends can be designed that can achieve similar goals as Charm, but for other target programming languages, e.g., C/C++, Rust or Go, allowing us to compare schemes in other metrics than Charm allows.

1.5 Paper Organization

We first introduce some preliminaries on which we build in Section 2. Then, we present our theoretical foundations in Section 3, on which our tool implemented in Section 4 relies. We demonstrate and evaluate our tool in Section 5 and finally conclude in Section 6. The code for this paper can be found on GitHub [AVAG].

2 Preliminaries

Notation. We use Σ^+ to denote the set of non-empty character strings over alphanumeric characters. We use $\llbracket \dots \rrbracket$ to denote (possibly empty) ordered lists. We use boldfaced variables $\mathbf{A}$ and $\mathbf{v}$ for matrices and vectors, respectively, where $(\mathbf{A})_{i,j}$ denotes the entry of $\mathbf{A}$ in the i -th row and j -th column. We define $\mathbf{1}_{i,j}^{d_1 \times d_2} \in \mathbb{Z}_p^{d_1 \times d_2}$ as the matrix with 1 in the i -th row and j -th column, and 0 everywhere else, and similarly $\mathbf{1}_i^{d_1}$ and $\bar{\mathbf{1}}_i^{d_2}$ as the row and column vectors with 1 in the i -th entry and 0 everywhere else.

2.1 Attribute-Based Encryption

We introduce some concepts related to ABE in this subsection.

Attribute Universes. The overarching set of all attributes $\mathcal{U}$ that can be used in a scheme is called the universe of attributes. The universe can be small or large. The universe is small, if the universe is bounded upon setup. In practice, the universe is small if the size of the master public key depends on the universe. In large-universe schemes, the universe is practically unbounded, and the public-key size does not depend on the universe.

Access Structures. We represent access policies $\mathbb{A}$ using linear secret sharing scheme (LSSS) matrices, which support monotone Boolean formulas [Bei96, GPSW06b].

Definition 1 (Access structures represented by LSSS [GPSW06b]). An access structure is represented as a pair $\mathbb{A} = (\mathbf{A}, \rho)$ such that $\mathbf{A} \in \mathbb{Z}_p^{n_1 \times n_2}$ is an LSSS matrix, where $n_1, n_2 \in \mathbb{N}$, and ρ is a function that maps its rows to attributes in the universe $\mathcal{U}$. Then, for some vector $\mathbf{v} = (s, v_2, \dots, v_{n_2}) \in_R \mathbb{Z}_p^{n_2}$, the i -th share of secret s generated by this matrix is $\lambda_i = \mathbf{A}_i \mathbf{v}^\top$, where $\mathbf{A}_i$ denotes the i -th row of $\mathbf{A}$. In particular, if $\mathcal{S}$ satisfies $\mathbb{A}$, then there exist a set of rows $\Upsilon = \{i \in [n_1] \mid \rho(i) \in \mathcal{S}\}$ and coefficients $\varepsilon_i \in \mathbb{Z}_p$ for all $i \in \Upsilon$ such that $\sum_{i \in \Upsilon} \varepsilon_i \mathbf{A}_i = (1, 0, \dots, 0)$, and by extension $\sum_{i \in \Upsilon} \varepsilon_i \lambda_i = s$, holds.

Supporting Negations. We can support negations by including a map $\rho' : [n_1] \rightarrow \{0, 1\}$, where $\rho'(j) = 1$ if the literal associated with row j is positive and $\rho'(j) = 0$ if it is negative. Schemes supporting OT-type negations [OT10, TKN20, AT20, RVV24] require the labeling of attributes. This is supported by including a map $\rho_{\text{lab}} : [n_1] \rightarrow \mathcal{L}$ that maps the rows to values in the label universe $\mathcal{L}$. To satisfy OT-type negations in the policy, the set must contain an attribute with the same label as the negated attribute, and it cannot have the same value as the negated attribute. The scheme also has a one-use restriction on the labels in the sets, i.e., each label may occur at most once. We call the unique attribute with the same label as the negated attribute the *alternative attribute*.

Access Policies with Deduplication. In some schemes, e.g., [AC17, Ven23b], an additional map $\tau : [n_1] \rightarrow [m]$ is introduced to the policy representation. This map is injective on the subset $\rho^{-1}(\text{attr}) \subseteq [n_1]$ for each attribute in $\rho([n_1])$ to which ρ maps the rows. Hence, for each $j, j' \in [n_1]$ with $j \neq j'$, we have $\tau(j) \neq \tau(j')$. Furthermore, τ is as efficient as possible, setting $m = \max_{\text{attr}} |\rho^{-1}(\text{attr})|$ to be the maximum number of occurrences of a single attribute in the policy. We will refer to τ as a deduplication map, as it essentially creates unique indices for each duplicated instance of the same attribute. Note that deduplication maps are often used to speed up decryption.

Pairings. We define a pairing to be an efficiently computable map e on three groups $\mathbb{G}, \mathbb{H}$ and $\mathbb{G}_T$ of prime order p , so that $e : \mathbb{G} \times \mathbb{H} \rightarrow \mathbb{G}_T$ maps two source groups $\mathbb{G}$ and $\mathbb{H}$ to the target group $\mathbb{G}_T$, with generators $g \in \mathbb{G}, h \in \mathbb{H}$ is such that (i) for all $a, b \in \mathbb{Z}_p$, it

holds that $e(g^a, h^b) = e(g, h)^{ab}$ (bilinearity), and (ii) for $g^a \neq 1_{\mathbb{G}}, h^b \neq 1_{\mathbb{H}}$, it holds that $e(g^a, h^b) \neq 1_{\mathbb{G}_T}$, where $1_{\mathbb{G}_T}$ denotes the unique identity element of the associated group $\mathbb{G}_T$ (non-degeneracy). We use PGGen to denote an algorithm that, on input the security parameter λ , returns a pairing group $\mathcal{PG} = (\mathbb{G}, \mathbb{H}, \mathbb{G}_T, p, e, g, h)$. We use the implicit representation used for group elements [EHK⁺13], i.e., $[x]_{\mathbb{G}'}$ denotes the element $(g')^x$, where $g' \in \mathbb{G}$ is the generator of some group $\mathbb{G}' \in \{\mathbb{G}, \mathbb{H}, \mathbb{G}_T\}$.

2.1.1 Syntax for ABE

We define ABE similarly as in [Att14, AC17], i.e., for general predicates. To improve practicality, we define the key-encapsulation version of the scheme.

Predicate Family. A predicate $P: \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ is a map over the ciphertext attribute space $\mathcal{X}$ and the key attribute space $\mathcal{Y}$ such that, for $x \in \mathcal{X}, y \in \mathcal{Y}$, $P(x, y) = 1$ if and only if the predicate evaluates to true for x and y .

Definition 2 (Attribute-based encryption (ABE) [AC17]). An attribute-based encryption scheme for a predicate $P: \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ consists of four algorithms:

- $\text{Setup} \rightarrow (\text{MPK}, \text{MSK})$: This probabilistic algorithm generates the master public key MPK and the master secret key MSK.
- $\text{KeyGen}(\text{MSK}, y) \rightarrow \text{USK}_y$: On input the master secret key MSK and some $y \in \mathcal{Y}$, this probabilistic algorithm generates a secret key USK_y .
- $\text{Encrypt}(\text{MPK}, x) \rightarrow (\text{CT}_x, K)$: On input the master public key MPK and $x \in \mathcal{X}$, this probabilistic algorithm generates a ciphertext CT_x and an encapsulated key K .
- $\text{Decrypt}(\text{MPK}, \text{USK}_y, \text{CT}_x) \rightarrow K$: On input the master public key MPK, secret key USK_y , and ciphertext CT_x , if $P(x, y) = 1$, then it returns K ; and otherwise, $\perp$.

Correctness. For all $x \in \mathcal{X}$, and $y \in \mathcal{Y}$ such that $P(x, y) = 1$,

$$\Pr[(\text{MPK}, \text{MSK}) \leftarrow \text{Setup}; (\text{CT}_x, K) \leftarrow \text{Encrypt}(\text{MPK}, x) : \\ \text{Decrypt}(\text{MPK}, \text{KeyGen}(\text{MSK}, y), \text{CT}_x) = K] = 1.$$

Security. The definition for security against chosen-plaintext attacks for ABE ensures that users cannot collude, i.e., if they cannot individually satisfy a predicate, then they cannot collectively do this either. The relevant security definitions can be found in Appendix A.

Ciphertext-Policy ABE. We focus mostly on ciphertext-policy ABE. The key attribute space $\mathcal{Y}$ consists of all possible sets over the attributes, and the ciphertext attribute space $\mathcal{X}$ of all policies over the possible sets (that fit within a certain expressivity class).

Multi-Authority ABE In the multi-authority setting, the role of the authority is shared among multiple authorities. To this end, the attributes are not only represented by their value, but also by an authority. For the encryption algorithm, this change is merely implicit (in the input policy), but for the the setup and key generation algorithm, we require a syntactical change. Specifically, each authority has its own setup, in which their key material is generated. Furthermore, each authority runs its own separate instance of the key generation algorithm. To tie the keys of one user together, we also require a global identifier as input: GID . In the constructions, the GID connects the keys together, so that they cannot be combined among users. Lastly, note that the attributes are represented differently in multi-authority ABE. In the CP-ABE setting, the attributes in the set are

represented as tuples, in which one entry is the authority. For the policies, which are represented by an linear secret sharing scheme (LSSS) matrix and row mappings, we include an additional mapping that maps the rows to authority identifiers.

2.2 Pair Encodings

Our work builds on the pair encodings framework [Att14, Wee14, AC17, Ven23b, RVV24], which considers a commonly used abstraction of many pairing-based ABE schemes. The most broadly applicable classes of pair encodings are covered by the Ven23 [Ven23b] and ISABELLA [RVV24] compilers. These classes are denoted by PES-Ven23 and PES-ISA, respectively. Because the difference between PES-Ven23 and PES-ISA is mostly syntactical, we consider the more compact definition of generalized pair encodings schemes in [Ven23b]. Because this class is contained in PES-ISA, we can use the PES-ISA compiler.

At a high level, pair encodings describe the exponent space of the associated pairing-based ABE scheme in vectors. The exponent space of the public keys are described by the master-key and common variables $\alpha = (\alpha_1, \dots)$ and $\mathbf{b} = (b_1, \dots)$, respectively. The secret keys are described by non-lone variables $\mathbf{r} = (r_1, \dots)$ and key polynomials $\mathbf{k} = (k_1, \dots)$. The ciphertexts are described by non-lone variables $\mathbf{s} = (s, s_1, \dots)$, ciphertext polynomials $\mathbf{c} = (c_1, \dots)$ and primed ciphertext polynomials $\mathbf{c}' = (c'_1, \dots)$. Decryption is expressed as matrices that describe which non-lone variables are combined with which polynomials. For example, if $\mathbf{E} = (E_{i,j})_{i,j}$ is the matrix such that $\mathbf{sEk}^\top$ is part of the correctness of the PES-Ven23, then it implicitly describes decryption as follows: $\mathbf{sEk}^\top = \sum_{i,j} E_{i,j} s_i k_j$.

Definition 3 (Generalized pair encoding schemes from Ven23 (PES-Ven23)). A generalized pair encoding scheme $\Gamma_{\text{PES-Ven23}}$ for the class of schemes covered by the Ven23 and ISABELLA compilers, and for a predicate $P: \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ and prime number p , with optionally some additional parameters par , is given by four deterministic polynomial-time algorithms as described below.

- $\text{Param}(\text{par}) \rightarrow (n_\alpha, n_b, \alpha, \mathbf{b})$: On input parameters par , the algorithm outputs $n_\alpha, n_b \in \mathbb{N}$ that specify the number of master key variables and common variables, respectively, which are denoted as $\alpha = (\alpha_1, \dots, \alpha_{n_\alpha})$ and $\mathbf{b} = (b_1, \dots, b_{n_b})$, respectively.
- $\text{EncKey}(y, \alpha, \mathbf{b}) \rightarrow (m_1, m_2, \mathbf{k}(\mathbf{r}, \hat{\mathbf{r}}, \alpha, \mathbf{b}, y))$: On input $y \in \mathcal{Y}$, α and $\mathbf{b}$, this algorithm outputs a vector of polynomials $\mathbf{k} = (k_1, \dots, k_{m_3})$, with $m_3 \in \mathbb{N}$, defined over non-lone variables $\mathbf{r} = (r_1, \dots, r_{m_1})$ and lone variables $\hat{\mathbf{r}} = (\hat{r}_1, \dots, \hat{r}_{m_2})$. Specifically, the polynomial k_i is expressed as

$$k_i = \sum_{j \in [n_\alpha]} \delta_{i,j} \alpha_j + \sum_{j \in [m_2]} \hat{\delta}_{i,j} \hat{r}_j + \sum_{j \in [m_1], k \in [n_b]} \delta_{i,j,k} r_j b_k,$$

for all $i \in [m_3]$, where $\delta_{i,j}, \hat{\delta}_{i,j}, \delta_{i,j,k} \in \mathbb{Z}_p$.

- $\text{EncCt}(x, \alpha, \mathbf{b}) \rightarrow (w_1, w_2, w'_2, c_M, \mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}, x), \mathbf{c}'(\mathbf{s}, \tilde{\mathbf{s}}, \alpha, x))$: On input $x \in \mathcal{X}$, α and $\mathbf{b}$, this algorithm outputs a blinding variable c_M and two vectors of polynomials $\mathbf{c} = (c_1, \dots, c_{w_3})$ and $\mathbf{c}' = (c'_1, \dots, c'_{w'_2})$, with $w_3, w'_2 \in \mathbb{N}$, defined over non-lone variables $\mathbf{s} = (s_1, s_2, \dots, s_{w_1})$, lone variables $\hat{\mathbf{s}} = (\hat{s}_1, \dots, \hat{s}_{w_2})$ and special lone variables $\tilde{\mathbf{s}} = (\tilde{s}_1, \dots, \tilde{s}_{w'_2})$. Specifically, the polynomial c_i is expressed as

$$c_i = \sum_{j \in [w_2]} \eta_{i,j} \hat{s}_j + \sum_{j \in [w_1], k \in [n_b]} \eta_{i,j,k} s_j b_k,$$

for all $i \in [w_3]$, where $\eta_{i,j}, \eta_{i,j,k} \in \mathbb{Z}_p$, the polynomial c'_i is expressed as

$$c'_i = \sum_{j \in [n_\alpha], j' \in [w_1]} \eta'_{i,j,j'} \alpha_j s_{j'} + \sum_{j \in [w'_2]} \hat{\eta}'_{i,j} \tilde{s}_j,$$

for all $i \in [w_4]$, where $\eta'_{i,j,j'}, \hat{\eta}'_{i,j} \in \mathbb{Z}_p$, and the variable c_M is expressed as

$$c_M = \sum_{j \in [w'_2]} \zeta_j \tilde{s}_j + \sum_{j \in [n_\alpha], j' \in [w_1]} \zeta_{j,j'} \alpha_j s_{j'}, \quad \text{where} \quad \zeta_j, \zeta_{j,j'} \in \mathbb{Z}_p$$

- $\text{Pair}(x, y) \rightarrow (\mathbf{e}, \mathbf{E}, \bar{\mathbf{E}})$: On input $x \in \mathcal{X}$, and $y \in \mathcal{Y}$, this algorithm outputs a vector $\mathbf{e} \in \mathbb{Z}_p^{w_4}$ and two matrices $\mathbf{E}$ and $\bar{\mathbf{E}}$ of sizes $w_1 \times m_3$ and $w_3 \times m_1$, respectively.

A PES-Ven23 is correct for every $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ such that $P(x, y) = 1$, it holds that $\mathbf{e}\mathbf{c}^\top + \mathbf{s}\mathbf{E}\mathbf{k}^\top + \mathbf{c}\bar{\mathbf{E}}\mathbf{r}^\top = c_M$.

Compact Notation. Because $\mathbf{e}, \mathbf{E}, \bar{\mathbf{E}}$ are typically sparse, we often only explicitly denote the nonzero entries, following [AC17, Att19]. This is done by using matrices $\mathbf{1}_{i,j}^{d_1 \times d_2}$ of dimension $d_1 \times d_2$, which consist of all zeros except in the entry in the i -th row and j -th column, which is 1. More generally, we can write $\mathbf{E} = \sum_{i \in [w_1], j \in [m_3]} (\mathbf{E})_{i,j} \mathbf{1}_{i,j}^{w_1 \times m_3}$

2.3 The Ven23 and ISABELLA Compilers

The Ven23 and ISABELLA compilers transform any PES-Ven23 for which the special selective symbolic property holds into a secure ABE scheme. These compilers allow much freedom in the distribution of the key and ciphertext components (as long as correctness is guaranteed). Furthermore, it allows common variables and non-lone key variables to be generated using a full-domain hash. Because the ISABELLA compiler can obtain the same schemes obtained with the Ven23 compiler (for the class of PES-Ven23), but under better security guarantees, we focus on the ISABELLA compiler in this work.

Difference Between Ven23 and ISABELLA. Compared to Ven23, the ISABELLA compiler has some practical advantages as well. For example, it uses the notion of split-predicate molds to support advanced functionalities such as multi-authority ABE in a more modular fashion than Ven23. Furthermore, it allows some of the public-key and ciphertext components that are instantiated in the target group to be moved to a source group, but this can only be done securely under certain conditions. Because both these properties go beyond the scope of this paper, we move the support for these to future work. Nevertheless, **pracy** is designed to take such future extensions into account.

Because all the schemes that we implement are ciphertext-policy ABE schemes, some of them in the single-authority setting and some in the multi-authority setting, we first give the simplest definition of the ISABELLA compiler (that supports single-authority schemes, abstracting away the split-predicate molds and the target-group conversions). After this, we show how the definition can be extended to the multi-authority setting.

Distribution of the Encodings. The Ven23 and ISABELLA compilers take as input explicit definitions for the distribution of the relevant encodings over $\mathbb{G}$ and $\mathbb{H}$ when they are instantiated. This distribution is denoted with the mapping $\mathfrak{D}$, and should ensure that the correctness of the PES is preserved, such that the correctness of the ABE scheme is also guaranteed. In particular, for the correctness of the decryption, we require that each pair of key and ciphertext encodings that needs to be paired has one input in $\mathbb{G}$ and one in $\mathbb{H}$. Furthermore, to ensure that encryption can be performed correctly, the master public keys required in computing a ciphertext encoding element need to be in the same group.

Full-Domain Hashes. When variables are generated implicitly by a full-domain hash (FDH), specified with the mapping $\mathcal{F}$, the description of the scheme needs to specify the hash input strings. The ISABELLA compiler also allows for explicit domain separation.

To ensure correctness of the scheme, we require the distribution over the $\mathbb{G}$ and $\mathbb{H}$ to be such that, for any common variable b_k that is provided implicitly by a hash, and each associated encoding k_i and c_i , it holds that they are placed in the same group. Similarly, we can define such a restriction for the other variables. Furthermore, if a non-lone variable and a common variable occur together in a product in one of the polynomials, then it cannot be the case that both are generated by an FDH.

Compatibility. We define the notion of compatibility in the context of the ISABELLA compiler that ensures that the inputs to the compiler are compatible for some PES-Ven23. This covers the correctness properties for the mappings $\mathfrak{D}$ and $\mathcal{F}$. Additionally, we require that the PES-Ven23 uses the same prime order p as the pairing group sampled in the compiler and that there exist full-domain hash functions into groups $\mathbb{G}$, $\mathbb{H}$.

Notation. In the compiler, we will run the algorithms of the PES-Ven23 and instantiate the polynomials in the groups by sampling randomly generated values in $\mathbb{Z}_p$ and compute group elements whose exponents correspond to the PES-Ven23 polynomials evaluated on these values. To simplify notation, we will use the same approach as in [RVV24], and use algorithms `SampleParam`, `SampleKey`, `SampleCt` to sample those values. We write, e.g., $(\alpha, \beta, \mathbf{b})_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleParam}()$, where $\mathcal{F}_1(\cdot) = 0$ indicates that we skip those variables that are instantiated with an FDH.

Definition 4 (The ISABELLA compiler for trivial molds). Let $\Gamma_{\text{PES-Ven23}}$ be a PES-Ven23 for predicate $P: \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ as defined in Definition 3 with distributions $\mathfrak{D}$ and $\mathcal{F}$, and let $\mathcal{PG}$ be a pairing group compatible with $\Gamma_{\text{PES-Ven23}}$. We assume the trivial mold $\text{SPM}[\Gamma_{\text{PES-Ven23}}]$, and define the scheme $\text{ABE-ISA} = \mathbf{Comp}[\mathcal{PG}, \Gamma_{\text{PES-Ven23}}, \text{SPM}[\Gamma_{\text{PES-Ven23}}], \mathfrak{D}, \mathcal{F}]$ as follows:

- `GlobalSetup` $\rightarrow$ `GP`: This algorithm picks full-domain hash functions $\mathcal{H}_{\mathbb{G}}: \{0, 1\}^* \rightarrow \mathbb{G}$ and $\mathcal{H}_{\mathbb{H}}: \{0, 1\}^* \rightarrow \mathbb{H}$ modeled as random oracles. It outputs global parameters $\text{GP} = (\mathcal{PG}, \mathcal{H}_{\mathbb{G}}, \mathcal{H}_{\mathbb{H}})$.
- `AuthoritySetup`($\mathcal{PG}$) $\rightarrow$ (`MPK`, `MSK`): On input the group description $\mathcal{PG}$, this algorithm runs $(\alpha, \mathbf{b})_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleParam}(\text{par})$, and outputs the master secret key $\text{MSK} = (\alpha, \{b_j \mid b_j \in \mathbf{b} \wedge \mathcal{F}_1(b_j) = 0\})$, along with master public key

$$\text{MPK} = (\{[\alpha_j]_{\mathbb{G}_T}\}_{\alpha_j \in \alpha}, \{[b_j]_{\mathfrak{D}(b_j)} \mid b_j \in \mathbf{b} \wedge \mathcal{F}_1(b_j) = 0\}).$$

- `KeyGen`(`MSK`, $\mathcal{S}$) $\rightarrow$ `USKS`: On input the master secret key `MSK` and a set $\mathcal{S}$, this algorithm generates $(\mathbf{k}(\mathbf{r}, \hat{\mathbf{r}}, \alpha, \mathbf{b}, \mathcal{S})) \leftarrow \text{EncKey}(\mathcal{S})$ and $(\mathbf{r}, \hat{\mathbf{r}})_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleKey}(\mathcal{S})$, and outputs the secret key

$$\text{USK}_{\mathcal{S}} = (\mathcal{S}, \{[r_j]_{\mathfrak{D}(r_j)} \mid r_j \in \mathbf{r} \wedge \mathcal{F}_1(r_j) = 0\}, \{[k_i]_{\mathfrak{D}(k_i)}\}_{k_i \in \mathbf{k}}).$$

- `Encrypt`(`MPK`, x) $\rightarrow$ (`CTx`, K): On input the master public key `MPK` and some policy x , this algorithm first generates $(w_1, w_2, w'_2, c_M, \mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}, x), \mathbf{c}'(\mathbf{s}, \hat{\mathbf{s}}, \alpha, x)) \leftarrow \text{EncCt}(x)$ and $(\mathbf{s}, \hat{\mathbf{s}}, \tilde{\mathbf{s}})_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleCt}(x)$, and outputs the key $K = e(g, h)^{c_M}$ and the ciphertext

$$\text{CT}_x = (x, [s]_{\mathfrak{D}(s)}, \{[s_j]_{\mathfrak{D}(s_j)} \mid j \in [w_1] \wedge \mathcal{F}_1(s_j) = 0\}, \{[c_i]_{\mathfrak{D}(c_i)}\}_{i \in [w_3]}, \{[c'_i]_{\mathbb{G}_T}\}_{i \in [w_4]}).$$

- `Decrypt`(`MPK`, `USKS`, `CTx`) $\rightarrow$ K : On input the master public key `MPK`, a secret key `USKS` for set $\mathcal{S}$ and a ciphertext `CTx` for policy x , if $x \models \mathcal{S}$, then this algorithm

first obtains $(\mathbf{e}', \mathbf{E}, \bar{\mathbf{E}}) \leftarrow \text{Pair}(x, y)$, sets

$$\begin{aligned} \mathfrak{P} = & \{(s_j, k_i, \mathbf{E}_{j,i}) \mid i \in [m_3], j \in \overline{[w_1]}, \mathbf{E}_{j,i} \neq 0 \wedge \mathfrak{D}(s_j) = \mathbb{G}\} \\ & \cup \{(k_i, s_j, \mathbf{E}_{j,i}) \mid i \in [m_3], j \in \overline{[w_1]}, \mathbf{E}_{j,i} \neq 0 \wedge \mathfrak{D}(s_j) = \mathbb{H}\} \\ & \cup \{(r_j, c_i, \bar{\mathbf{E}}_{i,j}) \mid i \in [w_3], j \in [m_1], \bar{\mathbf{E}}_{i,j} \neq 0 \wedge \mathfrak{D}(r_j) = \mathbb{G}\} \\ & \cup \{(c_i, r_j, \bar{\mathbf{E}}_{i,j}) \mid i \in [w_3], j \in [m_1], \bar{\mathbf{E}}_{i,j} \neq 0 \wedge \mathfrak{D}(r_j) = \mathbb{H}\}, \end{aligned}$$

and then retrieves

$$\prod_{i \in [w_4]} [c'_i]_{\mathbb{G}_T}^{\mathbf{e}'_i} \prod_{(l, \mathbf{r}, \mathbf{e}) \in \mathfrak{P}} e([l]_{\mathbb{G}}, [\mathbf{r}]_{\mathbb{H}})^{\mathbf{e}} = e(g, h)^{\mathbf{e}' \mathbf{c}'^T + \mathbf{s} \mathbf{E} \mathbf{k}^T + \mathbf{c} \bar{\mathbf{E}} \mathbf{r}^T} = e(g, h)^{c_M}.$$

The ISABELLA Compiler for the Multi-Authority Mold. We can adapt this definition to the multi-authority setting as follows. By taking an authority identifier as input to the AuthoritySetup, so that it can be run by each individual authority (rather than once for a single authority). Furthermore, the key generation algorithm takes as extra input the global identifier GID and a single authority's master secret key.

2.4 Access Structures in Symbolic-Property-Based Compilers

One subtle drawback of all compilers based on the symbolic property [AC17, Ven23b, RVV24] is the way that policies are represented in the keys and ciphertexts. Whereas in most traditional CP-ABE schemes, such as those by Waters (Wat11) [Wat11] and Rouselakis and Waters (RW13) [RW13], the secret shares λ_j generated with $\mathbb{A} = (\mathbf{A}, \rho)$ reconstruct s , where s is the non-lone variable used in masking the message, i.e., $M \cdot e(g, h)^{\alpha s}$. In the PESs corresponding to these schemes, see e.g., [Ven23a], the secret shares λ_j share sb , which has two notable drawbacks, related to the implementation of the secret sharing functionality and the efficiency.

Access Structures in Traditional Schemes. To illustrate the problem, we consider Wat11 as a more concrete example. In the original scheme (not instantiated using compilers), the policy shares are encoded in the ciphertext components of the form: $C_{1,j} = B^{\lambda_j} \cdot B_{\rho(j)}^{s_j}$, where B and $B_{\rho(j)}$ are part of the public key and s_j is generated at random during encryption and $\lambda_j = A_{j,1}s + \sum_{k \in [2, n_2]} A_{j,k}v_k$. Here, $(v_2, \dots, v_{n_2})$ is a randomly generated vector in $\mathbb{Z}_p^{n_2-1}$. In this form, computing B^{λ_j} is straightforward: the encrypting user can first compute λ_j (over $\mathbb{Z}_p$) and then perform one exponentiation.

Access Structures in Compiled Schemes. In the scheme compiled using a symbolic-property-based compiler, it is less straightforward. The problem is that ciphertext polynomial underlying the $C_{1,j}$ component does not satisfy the PES syntax, which would be $c_{1,j} = \lambda_j b + s_j b_{\rho(j)}$. In particular, if we write out $\lambda_j b = A_{j,1}sb + \sum_{k \in [2, n_2]} A_{j,k}v_k b$, then we see that the polynomial contains monomials of the form $A_{j,k}v_k b$. This is a product of a coefficient $A_{j,k}$, a *lone ciphertext variable* v_k and a common variable b . However, as the syntax of PES dictates, common variables can only occur in products with *non-lone variables*. (Note that we cannot simply make v_k non-lone, because then g^{v_k} would need to be included as part of the ciphertext, and this is insecure. We can also not readily allow products of lone and common variables in the keys and ciphertexts, because the symbolic property does not apply to such products.)

To circumvent this issue, we instead write the polynomial as $c_{1,j} = \lambda_j + s_j b_{\rho(j)}$, where $\lambda_j = A_{j,1}sb + \sum_{k \in [2, n_2]} A_{j,k}v_k$ is directly a share of sb instead of s . In this case, the polynomial does follow the syntax, because the monomials $A_{j,k}v_k$ are now products of coefficients $A_{j,k}$ and lone variables v_k . It is also correct, $\sum_{j \in \Upsilon, k \in [n_2]} \varepsilon_j A_{j,k}v_k = v_1$, where

$v_1 = sb$, which we initially also required. When compiled, the associated ciphertext component needs to compute $g^{\lambda_j} = g^{A_{j,1}sb + \sum_{k \in [2, n_2]} A_{j,k}v_k} = B^{A_{j,1}s} \cdot g^{\sum_{k \in [2, n_2]} A_{j,k}v_k}$, which requires two exponentiations instead of one. Encryption is thus inherently less efficient than in traditional schemes.

On the Generality of Secret Sharing Scheme Implementations. However, there is another – more subtle – issue related to the implementation of the policies. Specifically, it is related to the generality and applicability of the implementation. In the traditional version of the scheme, the shares are computed over $\mathbb{Z}_p$. Essentially any linear secret sharing scheme that computes the shares in a way that satisfies the structure in [Definition 1](#) suffices, e.g., the efficient encoding scheme in [\[LW10\]](#), those in [\[GPSW06b\]](#) based on Shamir’s secret sharing scheme [\[Sha79\]](#) and the secret sharing routine in [\[TKN20\]](#). Here, it is important to note that these schemes do not always explicitly compute $A_{j,k}$. In fact, the latter two secret sharing algorithms only compute $A_{j,k}$ implicitly, and rather compute λ_j by propagating secrets down an access tree that represents the access policy.

Although these algorithms have been shown to be equivalent (see, e.g., [\[LCW10, dIPVA22a\]](#)), the conversion algorithms that can be used to compute $A_{j,k}$ adds extra overhead and complexity to the implementation of algorithms. For example, for existing implementations, e.g., the Charm implementation of the secret sharing scheme associated with the built-in policy parser, we require a non-trivial adaptation of the code to obtain the matrix $\mathbf{A}$ instead of obtaining shares directly.

Converting Access Structures of Compiled Schemes to the “Traditional Way”. To benefit from the generality of secret sharing scheme implementations (and make these more interchangeable), we propose a conversion mechanism for the compiled schemes. We do this by leveraging the statistical indistinguishability of $A_{j,1}sb + \sum_{k \in [2, n_2]} A_{j,k}v_k$ (the PES-friendly version of the secret shares) and $A_{j,1}sb + \sum_{k \in [2, n_2]} A_{j,k}v_k b$ (the implementation-friendly version of the secret shares). These distributions are indistinguishable, because v_k only occur in combination with b in the latter. Because the distributions $\{(v_2, \dots, v_{n_2}) \mid v_k \in_R \mathbb{Z}_p\}$ and $\{(v_2 b, \dots, v_{n_2} b) \mid v_k \in_R \mathbb{Z}_p\}$ (with $b \in \mathbb{Z}_p$) are statistically indistinguishable, i.e., any tuple $(w_2, \dots, w_{n_2})$ has a probability of p^{-n_2+1} of being sampled, in both distributions. (This is under the assumption that $b \neq 0$, which happens with probability $\frac{p-1}{p}$.)

Although we cannot directly leverage this statistical argument by arguing that the PESs are statistically equivalent, we can use it to argue that the compiled schemes are statistically indistinguishable from one another.

Approximate Pair Encodings. We achieve this indistinguishability by defining the new notion of approximate pair encodings. These are an extension to “regular” pair encoding schemes in that we also allow products of lone variables with other variables in the key and ciphertext polynomials (i.e., common variables, but also master-key and semi-common variables). The associated security property captures a generalization of the statistical security argument we used to explain that the two different ways of computing the secret shares yield two schemes that are statistically indistinguishable. In this definition, an approximate PES is always related to an actual PES-Ven23 or PES-ISA, and in particular, the distribution of the approximate PES should be statistically close to that of the actual PES-Ven23 or PES-ISA. If this holds, the security of the compiled approximate PES is implied by the security of the compiled PES-Ven23 or PES-ISA. Our results about this can be found in [Appendix B](#).

3 Theoretical Foundations for pracy

In this section, we explain the conceptual design underlying our tool. To this end, we introduce a new formal model and type system to properly and automatically analyze scheme specifications and match the input to the format of the PES-Ven23.

As mentioned in the introduction, the central difficulty for our tool is dealing with universally quantified objects such as variables or polynomials. These quantifications must be retained, even in the code generation, because the sets over which we quantify are not fixed at compile time. For example, the policies are given as input to the scheme’s encryption *after* the scheme is implemented and not *before*. (If we were to “unroll” those quantifications, we would have to create a fresh implementation of the scheme for each set and policy for which we want to use the scheme, which is inefficient and unscalable.)

We begin by introducing our models and data types in [Section 3.1](#). Based on these, we develop a type system which we employ to realize correctness and consistency checks. Ultimately, we define notions of equivalence and inheritance of quantifications to illustrate the usage of our models.

3.1 Modeling Universally Quantified Objects

To model universally quantified objects, we need to identify and maintain the relation between names introduced by quantifications and indices referring to them. We start by looking at the sets and functions to build quantifications, i.e., the *producers* of names. After introducing the notion of Quants in [Definition 5](#), we continue with modeling Idxs ([Definition 6](#)) and their associated functions. These correspond to the *consumers* of names.

We refer to the following running examples to better illustrate our concepts.

$$\{\alpha_l\}_{l \in [n_{\text{aut}}]} \quad (1)$$

$$\{k_l = \alpha + r_l b_l\}_{l \in \tilde{\rho}_S(\mathcal{S})} \quad (2)$$

$$\{k_{2,\text{attr}} = \alpha_{\tilde{\rho}_S(\text{attr})} + r_{\text{GID}} b\}_{\text{attr} \in \mathcal{S}} \quad (3)$$

The first example in [Equation 1](#) is a set of master-key variables, quantified over the authorities. Next, [Equation 2](#) features a set of key polynomials, quantified over the set of authorities that follows from applying function $\tilde{\rho}_S(\text{attr})$ to the attribute set $\mathcal{S}$. Lastly, [Equation 3](#) again shows a set of key polynomials, quantified over the attributes in $\mathcal{S}$. However, the (master-key) variable α is indexed by an attribute mapped to its corresponding authority via $\tilde{\rho}_S(\text{attr})$. [Equation 3](#) also features the *special* variable r_{GID} .

Modeling Sets. A universal quantification introduces a name for an element of a specific set, e.g., $\text{attr} \in \mathcal{S}$. For ABE, there is only a small amount of sets used in the majority of schemes. Common examples include the attribute universe $\mathcal{U}$ or the set of labels $\mathcal{L}$. [Table 1](#) shows a list of these sets based on a literature survey (see, e.g., [\[VAH23\]](#) for a comprehensive list of schemes). We call each such a set a *QSet* and assign it a unique name that is given in the middle column. This allows us to give concise and independent definitions while unifying several works of literature. The explicit inclusion of LSSSRows and LINCOMB as unions of other QSets is purely for convenience. The bottom-most set DEDUPIDCS stems from a randomness-reuse optimization to reduce the number of pairings (cf. [Section 2.1](#)). We include this to highlight the flexibility of our tool to seamlessly handle specialized set constructions. We denote the set of all QSets as $\mathcal{Q}_S$.

We formalize the types of the elements in the QSets by introducing the notion of a *QType*. The third column of [Table 1](#) lists the corresponding type for each QSet, which need not be unique among all QSets. The set of all possible QTypes is written as $\mathcal{Q}_T$.

Table 1: A list of commonly used sets for ABE scheme specification and their correspondence to our QSets and QTypes.

Literature	QSet	QType
$\mathcal{U}$	ATTRUNI	ATTR
$\mathcal{S}$	USERATTRS	ATTR
$[n_{\text{aut}}]$	AUTHS	AUTH
$\mathcal{L}$	LABELS	LBL
$\Psi = \{j \in [n_1] \mid \rho'(j) = 1\}$	PosLSSSRows	LSSSRow
$\bar{\Psi} = [n_1] \setminus \Psi$	NEGLSSSRows	LSSSRow
$\Psi \cup \bar{\Psi} \text{ or } [n_1]$	LSSSRows	LSSSRow
Υ	PosLINCOMB	LSSSRow
$\bar{\Upsilon}$	NEGLINCOMB	LSSSRow
$\Upsilon \cup \bar{\Upsilon}$	LINCOMB	LSSSRow
$\tau(\Psi \cup \bar{\Psi}) = \{\tau(j) \mid j \in \Psi \cup \bar{\Psi}\}$	DEDUPIDCS	DEDUPIDX

Modeling Global Maps. Quantifications may also contain mappings, such as $\tilde{\rho}_{\mathcal{S}}(\text{attr}): \mathcal{S} \rightarrow [n_{\text{aut}}]$ in $l \in \tilde{\rho}_{\mathcal{S}}(\mathcal{S})$. Similarly to the QSets, we scan existing literature and compile a list of mappings required. They are given in Table 2. Each mapping has an explicit name (second column) and maps elements of one type, i.e., the *domain type*, to elements of another type, i.e., the *codomain type*. We use $\mathcal{Q}_M$ to refer to the set of all QMaps and id , where id denotes the identity function.

Table 2: A list of commonly used mappings for ABE scheme specification their correspondence to our QMaps and the QType of their respective domain and codomain.

Literature	QMap	Domain	Codomain
$\phi: \mathcal{S} \rightarrow \mathcal{L}$	ATTRToLBL	ATTR	LBL
$\tilde{\rho}_{\mathcal{S}}: \mathcal{S} \rightarrow [n_{\text{aut}}]$	ATTRToAUTH	ATTR	AUTH
$\tilde{\rho}: [n_1] \rightarrow [n_{\text{aut}}]$	LSSSRowToAUTH	LSSSRow	AUTH
$\rho_{\text{lab}}: [n_1] \rightarrow \mathcal{L}$	LSSSRowToLBL	LSSSRow	LBL
$\rho: [n_1] \rightarrow \mathcal{U}$	LSSSRowToATTR	LSSSRow	ATTR
$\tau: [n_1] \rightarrow [m]$	LSSSRowToDEDUPIDX	LSSSRow	DEDUPIDX

Modeling Quantifications. The combination of QSets and QMaps is the basis for our model of universal quantifications as defined below.

Definition 5 (Quant). A *Quant* q is a triple $(n, s, m) \in \Sigma^+ \times \mathcal{Q}_S \times \mathcal{Q}_M$. The constituents n , s and m are called *name*, *base set* and *global map* of q , respectively.

Intuitively, a Quant (n, s, m) directly corresponds to $\forall n \in m(s)$, except that we restrict the possible functions m and sets s to those typically used for ABE schemes. This is required, as our tool needs to have a semantic understanding of the quantifications at hand. The Quants for our introductory examples would be $(l, \text{AUTHS}, \text{id})$ for $\forall l \in [n_{\text{aut}}]$, $(l, \text{USERATTRS}, \text{ATTRToAUTH})$ for $\forall l \in \tilde{\rho}_{\mathcal{S}}(\mathcal{S})$ and $(\text{attr}, \text{USERATTRS}, \text{id})$ for $\forall \text{attr} \in \mathcal{S}$ respectively. The idea is now to associate each object to zero or more Quants to keep track of whether (and if so, how) the object is quantified.

Modeling Local Maps for Indices. We now model the indices, i.e., the (potential) consumers of the names of Quants. There are three different types we have to consider:

constant indices, such as $_2$ in Equation 3 and quantified indices like $_l$ in Equation 2. The latter can further be differentiated into plain and mapped indices. An example of a mapped index is $\bar{\rho}_S(\text{attr})$ in Equation 3. Note that our notion of *constant* and *quantified* corresponds to free and bound variables in other fields such as Lambda Calculus.

Table 3: A list of commonly used mappings for ABE scheme specifications and their correspondence to our IMaps and the QType of their respective domain and codomain.

IMap	Domain(s)	Codomain
TOATTR	LSSSRow	ATTR
TOAUTH	ATTR, LSSSRow	AUTH
TOLBL	ATTR, LSSSRow	LBL
TODEDUPIDX	LSSSRow	DEDUPIDX
TOXATTR	ATTR, LSSSRow	ATTR
TOALTATTR	LSSSRow	ALTATTR

To this end, we introduce IMaps to model the mappings applied to individual indices. Table 3 shows the IMaps supported by our tool. The notational conventions for these types of mappings are inconsistent in literature. For instance, TOATTR is typically written as $\rho(\cdot)$ following the functional notation of QMaps in Table 2. However, for TOXATTR a new variable is introduced using the argument as its index, i.e., x_{attr} . We consolidate these differences under a uniform and consistent representation. IMaps may have multiple domains. The specific choice of domain can always be uniquely determined from the quantification. We use $\mathcal{I}_M$ to refer to the set of all IMaps and the identity function.

Modeling Indices. Indices provide the *only* way to “consume” the names introduced by Quants. This simplifies the design significantly and poses no restriction in practice. An Idx is defined as a name with a map as follows.

Definition 6 (Idx). An Idx $i = (n, m) \in \Sigma^+ \times \mathcal{I}_M$ is a pair consisting of an identifier n and an IMap m called *name* and *local map* respectively.

An Idx in conjunction with one or more Quants has a special semantic meaning. If the name of a Quant matches the name of an Idx, the Idx (and by extension the object it is associated with) is considered to be *quantified*. Conversely, if there is no matching name in the quantification, the index is constant. Using our model, the $_l$ from our example would simply be (l, id) . Similarly, $_2$ is represented as $(2, \text{id})$ and $\bar{\rho}_S(\text{attr})$ as $(\text{attr}, \text{TOAUTH})$.

Modeling Variables. Combining all aforementioned concepts, we define quantified variables as follows.

Definition 7 (Var). A Var v is a triple $(n, i, q) \in \Sigma^+ \times [\text{Idx}] \times [\text{Quant}]$ consisting of a *name*, *indices* and *quantifications* in this order.

According to the above definition, our introductory example in Equation 1 is internally represented as a Var as follows:

$$(\text{alpha}, [(l, \text{id})], [(l, \text{AUTHS}, \text{id})]) \quad (4)$$

Note that for simpler processing we disallow Greek letters and spell out the name *alpha* in ASCII. In general, names can be chosen freely. Similarly, the name l that appears in the Idx and Quant is not restricted as long as it is identical in both places.

Modeling Polynomials. We similarly define polynomials as follows.

Definition 8 (Poly). A *Poly* v is a quadruple $(n, i, q, b) \in \Sigma^+ \times \llbracket \text{Idx} \rrbracket \times \llbracket \text{Quant} \rrbracket \times \star$ where $\star$ denotes the set of arithmetic expressions over integers and Vars. We call n the *name*, i the *indices*, q the *quantifications* and b the *body* of a Poly.

We transparently consider Polys as Vars by relying on the canonical mapping from Poly to Var which simply ignores the body. Equation 5 and Equation 6 show our internal representation of the key polynomials in the examples at the beginning of this section.

$$(k, \llbracket (1, \text{id}) \rrbracket, \llbracket (1, \text{USERATTRS}, \text{ATTRTOAUTH}) \rrbracket, \left(\begin{array}{c} \text{alpha} \\ \left(\begin{array}{c} \oplus \\ \left(\begin{array}{c} r \\ \llbracket (1, \text{id}) \rrbracket \end{array} \end{array} \right) \otimes \left(\begin{array}{c} b \\ \llbracket (1, \text{id}) \rrbracket \end{array} \end{array} \right) \end{array} \right) \right) \quad (5)$$

Note how the variables occurring in the body of the polynomials are *unquantified*. Using Quants inside the body of a Poly is not allowed as this is semantically wrong. Instead, all Vars *inherit* the Quants of the Poly in which they are contained. The variable $\langle \text{rgid} \rangle$ is built into our tool and treated specially (cf. Section 3.3). The names of all other Vars, Idxs and Quants can be chosen freely.

$$(k, \llbracket (2, \text{id}), (\text{attr}, \text{id}) \rrbracket, \llbracket (\text{attr}, \text{USERATTRS}, \text{id}) \rrbracket, \left(\begin{array}{c} \text{alpha} \\ \left(\begin{array}{c} \oplus \\ \left(\begin{array}{c} \text{attr}, \text{ToAUTH} \\ \llbracket \rrbracket \end{array} \right) \otimes \left(\begin{array}{c} \langle \text{rgid} \rangle \\ \left(\begin{array}{c} \oplus \\ \left(\begin{array}{c} b \\ \llbracket \rrbracket \end{array} \end{array} \right) \end{array} \right) \end{array} \right) \end{array} \right) \right) \quad (6)$$

3.2 Processing Universally Quantified Objects

After detailing our data model, we now illustrate how we work with Vars and Polys internally. To this end, we discuss correctness checks to ensure well-definedness of all objects. Then, the main focus is on carving out suitable notions of “equality” to connect multiple occurrences of the same Vars. For ease of readability, everything referring to Vars only can be applied analogously to Polys as well.

Validity and Type Correctness. The definitions above allow the constructions of Vars which are not sensible (or even well-defined). To process these objects we need to check their validity, which is defined as follows.

Definition 9 (Validity, Valid Variable). Let $v = (n, \llbracket i_1, i_2, \dots, i_n \rrbracket, \llbracket q_1, q_2, \dots, q_m \rrbracket)$ be a Var. We say v is *valid*, if and only if the following hold:

- (i) $\forall i, j \in [m]$ such that $i \neq j$, the names of q_i and q_j are different and
- (ii) $\forall i \in [m]$ there is at least one $k \in [n]$ such that q_i and i_k have matching names and
- (iii) the local map of each constant Idx i_k for $k \in [n]$ is id.

Intuitively, the above definition prevents three potential sources of errors. First, duplicate quantifications like $\forall n \in S, \forall n \in S' : \text{expr}(n)$ are excluded. Second, all names that are introduced in the quantifications are consumed at least once. Although this is not strictly ill-defined, this is likely a mistake by the user and could lead to unintended collisions in our backend (see Section 3.2). Third, a constant Idx with a local map other than id is not sensible, as constants are essentially untyped and local maps are typed.

Building on the validity of Vars, we can now uniquely derive types for quantified Idxs and introduce a more advanced notion of type correctness.

Definition 10 (Type Correct, Typechecks). Let v be a valid Var with indices $i = \llbracket i_1, i_2, \dots, i_n \rrbracket$ and quantifications $q = \llbracket q_1, q_2, \dots, q_m \rrbracket$. We say v is *type correct* (or *type-checks*) if and only if each quantified Idx i_k for $k \in [n]$ typechecks under the Quants q . Now, let $i' = (n, m_l)$ be such a quantified Idx in i , and let $q' = (n, s, m_g)$ be the unique Quant in q with a matching name. We say that i' *typechecks* under the Quants q if and only if the following hold:

- (i) $\text{dom}(m_g) = s$, i.e., the domain of the global map is the base set and
- (ii) $\text{codom}(m_g) = \text{dom}(m_l)$, i.e., the composition $m_l \circ m_g$ is well-defined.

We further define the QType of elements in $\text{codom}(m_l)$ as the *inferred type of an Idx*.

The above definition is a straightforward adaption of typing rules for function application in (typed) Lambda Calculus and functional programming. In particular, we reduce type correctness of variables to the type correctness of all quantified Idxs. The purpose of our type system is twofold. First, tracking the types is technically *required* to support backends written in typed languages such as C++ or Rust (cf. Section 4.2). Second, it allows additional correctness checks. Such correctness checks correspond directly to well-definedness of ABE scheme specifications in literature. By explicitly modeling Quants and Idxs using a type system, we are the first to enable fully automated correctness checks.

Inheritance of Quantifications. The variables contained inside the body of polynomials are not quantified explicitly. Instead, they inherit *all* quantifications from their surrounding object whenever we process them. Concretely, this means that $(r, \llbracket (l, \text{id}) \rrbracket, \llbracket \rrbracket)$ in Equation 5 is stored as $(r, \llbracket (l, \text{id}) \rrbracket, \llbracket (l, \text{USERATTRS}, \text{ATTRTOAUTH}) \rrbracket)$ in the set of non-lone key variables. This is required to model the correct semantic interpretation, which would generate a new random value for each authority referenced in the user’s attribute set¹.

Equivalence. Another problem is that a Var may occur in the PES in different representations, which are nontrivial to detect in an automated tool. For example, one would expect that $\{\alpha_l\}_{l \in [n_{\text{aut}}]}$ and $\{\alpha_x\}_{x \in [n_{\text{aut}}]}$ specify the same master-key variable. This is similar to what is known as *alpha equivalence* in Lambda Calculus. Further, the combinations of global and local maps may result in different representations. For example, suppose that Equation 1 and Equation 3 are part of a single specification. Then, α_l and $\alpha_{\tilde{p}_S(\text{attr})}$ can refer to the same objects, i.e., the key polynomial $k_{2,\text{attr}}$ “contains” (some of) the $\{\alpha_l\}_{l \in [n_{\text{aut}}]}$. To model these relationships, we define an equivalence relation for Vars.

Definition 11 (Equivalence of Vars). Two Vars $v = (n_v, i_v, q_v)$ and $w = (n_w, i_w, q_w)$ are *equivalent* if and only if all of the following hold:

- (i) the names are identical, i.e., $n_v = n_w$,
- (ii) the number of indices is identical, i.e., $\|i_v\| = \|i_w\|$ and
- (iii) all indices $i_v^{(k)}$ in i_v and $i_w^{(k)}$ in i_w for $k \in \llbracket \|i_v\| \rrbracket$ are either both
 - (a) not quantified and identical or
 - (b) typecheck and their inferred types are identical.

Note that this is an equivalence relation respecting reflexivity, symmetry and transitivity.

¹One could, in theory, turn things around and instead define Quants to be (polymorphic) containers that “enclose” variables and polynomials. Our notion of inheritance would then correspond to applying functions to the contained objects while keeping the container unchanged. Overall, we believe that this gives rise to a monadic structure.

Implication of Equivalence. Intuitively, our equivalence definition only considers the inferred QType of quantified Idxs while “ignoring” additional information such as the concrete name. This allows us to track possible renamings in the indices (also known as alpha conversions), and use different combinations of global and local maps while still being able to determine which variables are equivalent. In the examples below, we show how the different representations of variables in the PES can be “masked” by replacing quantified indices (and their associated quantifications) by their inferred type, e.g.,

$$\begin{aligned}
(b, \llbracket (2, \text{id}), (1, \text{id}) \rrbracket, \llbracket (1, \text{AUTHS}, \text{id}) \rrbracket) &\rightarrow (b, \llbracket 2, \text{AUTH} \rrbracket) \\
(b, \llbracket (2, \text{id}), (x, \text{ToAUTH}) \rrbracket, \llbracket (x, \text{ATTRUNI}, \text{id}) \rrbracket) &\rightarrow (b, \llbracket 2, \text{AUTH} \rrbracket) \\
(b, \llbracket (2, \text{id}), (x, \text{id}) \rrbracket, \llbracket (1, \text{USERATTRS}, \text{ATTRToAUTH}) \rrbracket) &\rightarrow (b, \llbracket 2, \text{AUTH} \rrbracket) \\
(b, \llbracket (2, \text{id}), (x, \text{id}) \rrbracket, \llbracket (1, \text{LABELS}, \text{id}) \rrbracket) &\rightarrow (b, \llbracket 2, \text{LBL} \rrbracket)
\end{aligned} \tag{7}$$

This allows us to determine efficiently which representations are equivalent.

Our definition of equivalence yields a relatively fine-grained partition on the set of all possible Vars. It also corresponds to the mental model of theoreticians and practitioners. A concrete example for this can be found in Definition 19 in [Ven23b]. Here, common variables $\{b_l\}_{l \in [n_{\text{aut}}]}$ as well as $\{b_{\text{attr}}\}_{\text{attr} \in \mathcal{U}}$ are defined. Occurrences of these, such as $\{\dots b_{\tilde{\rho}_S(\text{attr})} \dots\}_{\text{attr} \in \mathcal{S}}$ can only be understood correctly by tracking the “types”. Otherwise, this specification is ill-defined. For humans this is not a problem, as they can intuitively correct this. For our tool, however, we detect and handle such cases using equivalence.

Similarity. As our examples illustrate, equivalence is crucial in detecting different representations of the same variables. It, however, also illustrates that two different representations may not be equivalent, but very similar in a way that can cause issues when implemented. For example, in $\{b_l\}_{l \in [n_{\text{aut}}]}$ and $\{b_{\text{attr}}\}_{\text{attr} \in \mathcal{U}}$, the sets $\mathcal{U}$ and $[n_{\text{aut}}]$ may not be disjoint. They could thus yield the same public-key component during look-up and, in some circumstances, lead to confusions or insecurity. We model this through the notion of similarity.

Definition 12 (Similarity of Vars). Two Vars $v = (n_v, i_v, q_v)$ and $w = (n_w, i_w, q_w)$ are *similar* if and only if all of the following hold:

- (i) the names are identical, i.e., $n_v = n_w$,
- (ii) the number of indices is identical, i.e., $\|i_v\| = \|i_w\|$ and
- (iii) all indices $i_v^{(k)}$ in i_v and $i_w^{(k)}$ in i_w for $k \in [\|i_v\|]$ are either both
 - (a) both not quantified and identical or
 - (b) at least one is quantified.

Similarity only differs from equivalence in (iii)(b). Here, the inferred QType is no longer used as a distinguishing feature, allowing collisions across types and constants. The difference between equivalence and similarity is as follows. Equivalence is what we use internally to recognize the same variables in different forms. Similarity is what we check *additionally* to avoid unintended confusions in the specification or the generated code. [Appendix C](#) features a detailed example of how similarity can prevent subtle errors.

3.3 Special Variables

In the compiled ABE scheme, most variables in the PES are generated at random, however, some of them have a strictly specified semantic meaning. We call these variables *special variables*. Common examples include r_{GID} , a secret (often called s or $\tilde{s}$) that is shared via λ_j . Others are more delicate, such as the alternative attribute XATTRALT (which we defined in the context of OT-type negations ([Section 2.1](#))), for which no typical notation

exists in literature. Our approach to handle special variables is to insert a function call in the backend that retrieves the corresponding value. It is the backend’s responsibility to correctly implement this function. We currently only support a fixed set of special variables that are hard-coded into our tool. Our selection of special variables is based again on established literature (see Table 4 for all supported special variables).

Table 4: A list of all supported special variables and their typical notation in literature.

Literature	Special Variable	Description
r_{GID}	RGID	Random identifier to avoid collusion attacks
$s, \tilde{s}$	SECRET	The secret that is shared
λ_j	LAMBDA	Shares of the secret s or $\tilde{s}$ acc. to the policy
μ_j	MU	Shares of 0 acc. to the policy
ε_j	EPS	Coefficients to reconstruct the secret
x_{att}	XATTR	Numeric representation of attr in $\mathbb{Z}_p$
—	XATTRALT	Numeric representation of “alternative” attribute used for OT-type negation

4 Our Tool: pracy

In this section, we explain the design of the input and output format of our tool, **pracy**, and how it works internally. The ideas developed in Section 3 are applied to our practical implementation and challenges therein are discussed. Finally, we present our backends, their purpose and functionality and how they interact with our tool.

4.1 JSON Syntax

We begin by introducing an accessible input format based on JSON and L^AT_EX, which is easy to process yet intuitive and close to mathematical notations. We also base our syntax on existing scheme specifications from literature, e.g., [AC17, Att19, Ven23b, RVV24]. In the following, we explain the individual constituents and their representation as input to our tool. At a high level, the input fields for our JSON format are closely related to what we require as input for the Ven23 and ISABELLA compilers: the description of the PES-ISA and the associated group and FDH assignment functions. Furthermore, there is an additional field for meta data, which is relayed by the compiler to the backend.

Quantifications. A central component of the ABE scheme specifications are universal quantifications of the form $\forall x \in Y$. We indicate the quantification of an object by appending `_[var:SET]` to its textual form. The identifier **var** is the name of the quantified variable and can be chosen freely. The **SET** components must be one of several predefined *base sets* built into the compiler such as **USER_ATTRS** or **AUTHS**. Optionally, the quantification may contain a *global map* like `_[var:func(SET)]`, which corresponds to $\forall x \in f(Y)$. Again, **func** must be a function name known to the compiler.

Variables and Indices. The scheme specifications contain many different variables with and without indices. Variables are simply non-empty strings of lowercase letters optionally followed by `'` to model primes. Simple examples are **alpha** and **b'**. The indices are indicated using a trailing `_{1, att}` syntax just like in L^AT_EX. Each index can be either a number or an identifier. Whether an index is constant or refers to a quantified name depends on the quantifications that enclose a variable’s occurrence. Optionally, indices

may have a *local map*, like `_ {1, rho(x)}`, which corresponds to the mathematical notation $v_{1,\rho(i)}$. In this case, the variable x should be quantified and represent something that can be mapped to an attribute. Again, the allowed local maps are built into our tool.

Master-Key and Common Variables. A combination of quantifications and indexed variables is sufficient to represent the key components of the Param algorithm. For instance, the vector of common variables can be defined as $\mathbf{b} = (b, \{b_{\text{attr}}\}_{\text{attr} \in \mathcal{U}})$. For our tool, we specify this as a JSON array consisting of `b` and `b_{attr}_{attr:ATTR_UNI}`. Note that the names `b` and `attr` are chosen arbitrarily whereas `ATTR_UNI` has a distinct meaning. This is analogous to the mathematical specification itself.

Arithmetic Expressions. For the components defined by the EncKey, EncCt and Pair algorithms, we need arithmetic expressions. We allow arbitrary nesting of additions, subtractions, multiplications and divisions as well as (indexed) variables and integers. The numeric literals and operations are interpreted over $\mathbb{Z}_p$. By allowing any expression, we make the tool more accessible and more broadly applicable. For example, many schemes with negations require fractions as coefficients [OSW07, Att19].

Special Variables. Some variables, such as r_{GID} or x_{att} have a special meaning. To clearly indicate their special interpretation and avoid accidental name collisions we enclose *special variables* in angle brackets, e.g., `<rgid>`. Further, special variables can be considered as proper variables or as coefficients in the context of a polynomials, which is another reason why we distinguish them syntactically. The set of possible names is built into the compiler as they need to be treated specially (cf. Section 3.3). Just like regular variables, special variables may also have indices if their semantics allow this.

Polynomials. The key and ciphertext polynomials can be specified as a combination of quantifications, (indexed) variables and arithmetic expressions. Additionally, they are mapped to one of the groups $\mathbb{G}$, $\mathbb{H}$ or $\mathbb{G}_t$. By including the group directly in the specification of polynomials, we avoid the need of additional inputs, which may be out of sync. For example, for a polynomial $\{k_{1,l} = 2\alpha_l + r_{\text{GID}}b_l\}_{l \in [n_{\text{att}}]}$, our corresponding JSON format is

`(k_{1, 1} : G = 2*alpha_{1} + <rgid>*b_{1})_{[1:AUTHS]}`.

The $\mathbb{G}$ indicates that this polynomial is mapped to group $\mathbb{G}$. The expression references numeric constants, regular and special variables with the aforementioned syntax. Note that the quantification applies to the entire polynomial.

Pairings. The last algorithm of a PES-Ven23 is the Pair algorithm, which captures decryption. This is where we stray the furthest from the mathematical notation, because the matrix notation is difficult to capture in a user-friendly way. Instead, we use the more compact notation explained below Definition 3, e.g.,

$$\mathbf{E} = \dots + \sum_{j \in \Upsilon} \varepsilon_j \mathbf{1}_{(2,\tau(j)),(2,\rho(j))}^{\omega_1 \times m_3} + \dots$$

By definition, the matrix $\mathbf{E}$ defines the pairings of non-lone cipher variables $\mathbf{s}$ and key polynomials $\mathbf{k}$. The sum ranges over all $j \in \Upsilon$, which corresponds to all the rows of the LSSS matrix that are needed to reconstruct the secret given the user's set of attributes. The coefficient ε_j is the reconstruction coefficient of the j -th row. The indices $(2, \tau(j))$ and $(2, \rho(j))$ indicate which entries of $\mathbf{s}$ and $\mathbf{k}$ are paired. In essence, this can be rewritten as

$$\dots, \forall j \in \Upsilon : e(s_{2,\tau(j)}, k_{2,\rho(j)})^{\varepsilon_j}, \dots$$

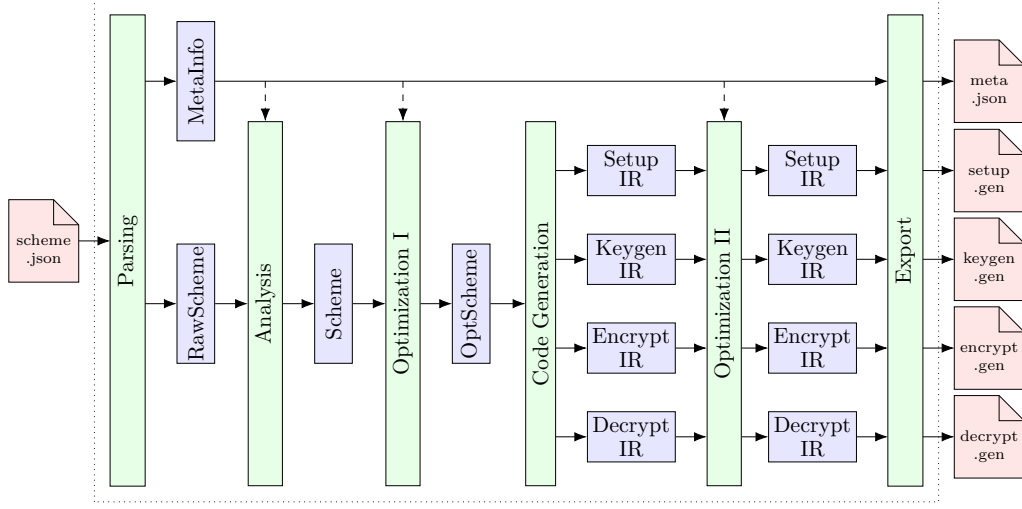


Figure 2: Overview of the data flow through our **pracy** tool. Blue items mark intermediate data structures, green is used to highlight functional steps and red indicates input and output.

which captures the same information. We exploit this correspondence to specify the decryption logic with the previously presented syntax for quantifications, variables and expressions. Concretely, the example at hand is written in our JSON syntax as

$$(s_{\{2, j.\text{to_dedup}\}} \sim k_{\{2, j.\text{attr}\}} = \langle \text{epsilon} \rangle_{\{j\}})_{[j:\text{POS_LSSS_ROWS}]}$$

where the right-hand side of the equals may be any expression including numeric constants. The syntax for $\overline{\mathbf{E}}$ is identical. We can categorize the inputs based on what variables and polynomials are referenced. The vector $\mathbf{e}$ is treated similarly, except that the left side of the equals sign is a single variable instead of a pair separated by a tilde.

Full-Domain Hash Specifications. Some variables can be computed by an FDH at different places instead of being passed as part of the MSK or the USK. The user can specify which variables should be produced by a full-domain hash (FDH) by including it in the `fdh_map` field. The syntax is simple and reuses the syntactical elements from above. For instance, the scheme specification may define $\mathcal{F}_1(r_l) = 2 \forall l \in [n_{\text{aut}}]$, i.e., r_l should be computed implicitly by hash function $\mathcal{H}_2$, e.g., $g^{r_l} = \mathcal{H}_2(l)$. In our JSON format, the same would be written as `r_{\{1\}}[1:AUTHS] # 2`.

4.2 Architecture of **pracy**

We intend our tool to serve as a basis for many future research endeavors. To this end, we designed **pracy** with modularity, extensibility and usability in mind. We implement **pracy** in Python, which is a widely spread and easy-to-use language. Figure 2 shows a schematic representation of the compiler. The dotted box encloses the internal design and data flow, whereas the red elements outside show the input and output data. The different processing steps are shown in green and will be discussed in detail in this section. Blue boxes represent intermediate data structures, which are produced and consumed sequentially, except for **MetaInfo**. This special object is extracted from the input and passed to the output unmodified. The dashed arrows indicate that future version can access additional input data we did not foresee to be useful or required.

Input and Output. The input to our tool is of a single JSON file consisting of the scheme specification and meta data. The meta data serves a dual purpose. First, it allows the user to provide additional inputs to `pracy`. Second, it captures inputs and configuration options for the backend (cf. [Section 4.5](#)). This is necessary, because ABE schemes offer a wide range of different features, such as support for negations, labels or multiple authorities. These features cannot always be inferred automatically, but a backend may need to adjust their computations based on this. A single file as input for each scheme allows for an easy and self-contained exchange.

Parsing and Export. Parsing the input file is straightforward using open-source parsing tool kits such as Lark [\[Lar\]](#). The scheme specification follows the syntax detailed in [Section 4.1](#). The export layer is crucial to keep backends independent. Within our tool, code is represented in our own IR. When exporting the generated code, each `IrStmt` is converted into the syntax expected by the selected backend. This step is highly flexible and allows for customization of variable and function names, function call syntax and even indentation. For instance, the statement `AddG(trgt, lhs, rhs)` is exported to

```
trgt = ops.add_g(lhs, rhs);    and    trgt = lhs + rhs
```

for our C++ and Python backends, respectively.

Analysis. The analysis phase is the heart of our practical contribution. It uses our model in [Section 3](#) to correctly analyze and “understand” the input. We also need to derive information not specified in the JSON input (e.g., the non-lone randoms), and verify the correctness of the group and FDH assignments. We explain this in detail in [Section 4.3](#).

Optimization I and II. The optimization passes are placeholders for future optimizations that we consider out of scope for this paper. Note that the two passes act on different data structures. The first optimization layer is designed for high-level changes such as altering key or cipher polynomials. The second layer has access to the generated code and can perform more implementation-oriented changes, such as peephole optimizations.

Code Generation. Code generation turns the internal representation of the ABE scheme at hand into a sequence of statements `IrStmt` in our intermediate representation. Most of this is straightforward and follows known techniques from compiler engineering. The most notable exception is the handling of our customized formal model and type system, including Quants and Idxs. We discuss code generation in more detail in [Section 4.4](#).

4.3 Analysis Phase

The analysis layer – which turns a raw JSON-based scheme specification into an internal representation suitable for automatic code generation – is the heart of our practical contribution. Numerous pitfalls and circular dependencies render this task unexpectedly difficult, up to the point where new theoretic foundations are required. [Algorithm 1](#) highlights the top-level analysis procedure. We use $\mathcal{R}_?$ to refer to components of the raw input, $\mathcal{M}_?$ to denote associate mappings and $\mathcal{V}_?$ for sets of (processed) variables. These maps and sets are custom data structures based on the equivalence of variables in [Definition 11](#). The empty lines separate the code into seven conceptual blocks.

Analyzing the Master-Key and Common Variables. We begin with establishing a *ground truth* consisting of the variant, and the master-key and common variables. The variant, i.e., whether the scheme is a CP-ABE or a KP-ABE scheme, can be heuristically inferred

Algorithm 1 The analysis step in *pracy*.

```

1: procedure ANALYZEScheme( $\mathcal{R} := \text{RawScheme}$ )
2:    $q_k, q_c := \text{ExtractQuants}(\mathcal{R}_{\text{kp}}, \mathcal{R}_{\text{cp}})$ 
3:    $v := \text{AnalyzeVariant}(q_k, q_c)$   $\triangleright$  CP-ABE vs. KP-ABE
4:    $\mathcal{M}_K := \emptyset$   $\triangleright$  Variable Kind Map
5:    $\mathcal{M}_G := \emptyset$   $\triangleright$  Variable Group Map
6:    $\mathcal{V}_{\text{msk}} := \text{AnalyzeMasterKeyVars}(\mathcal{M}_K, \mathcal{R}_{\text{msk}})$ 
7:    $\mathcal{V}_{\text{cv}} := \text{AnalyzeCommonVars}(\mathcal{M}_K, \mathcal{R}_{\text{cv}})$ 

8:    $\mathcal{V}_{\text{klr}} := \emptyset, \mathcal{V}_{\text{knlr}} := \emptyset$ 
9:    $\mathcal{V}_{\text{kp}} := \text{AnalyzeKeyPolys}(v, \mathcal{M}_K, \mathcal{M}_G, \mathcal{V}_{\text{klr}}, \mathcal{V}_{\text{knlr}}, \mathcal{R}_{\text{kp}})$ 

10:   $\mathcal{V}_{\text{clr}} := \emptyset, \mathcal{V}_{\text{cnlr}} := \emptyset, \mathcal{V}_{\text{cslr}} := \emptyset$ 
11:   $\mathcal{R}_{\text{cprim}}, \mathcal{R}_{\text{csec}}, \mathcal{R}_{\text{cblind}} := \text{CategorizeCipherPolys}(\mathcal{R}_{\text{cp}})$ 
12:   $\mathcal{V}_{\text{cprim}} := \text{AnalyzePrimaryCipherPolys}(v, \mathcal{M}_K, \mathcal{M}_G, \mathcal{V}_{\text{clr}}, \mathcal{V}_{\text{cnlr}}, \mathcal{R}_{\text{cprim}})$ 
13:   $\mathcal{V}_{\text{csec}} := \text{AnalyzeSecondaryCipherPolys}(v, \mathcal{M}_K, \mathcal{M}_G, \mathcal{V}_{\text{cnlr}}, \mathcal{V}_{\text{cslr}}, \mathcal{R}_{\text{csec}})$ 
14:   $\mathcal{V}_{\text{cblind}} := \text{AnalyzeBlindingPoly}(v, \mathcal{M}_K, \mathcal{V}_{\text{cnlr}}, \mathcal{V}_{\text{cslr}}, \mathcal{R}_{\text{cblind}})$ 

15:   $\mathcal{M}_{\text{fdh}} := \text{AnalyzeFDHMap}(\mathcal{M}_K, \mathcal{R}_{\text{fdh}})$ 

16:   $\mathcal{V}_{\text{kp}} := \text{PostAnalyzeKeyPolys}(\mathcal{V}_{\text{kp}}, \mathcal{M}_{\text{fdh}})$   $\triangleright$  Retroactively verify assumptions
17:   $\mathcal{V}_{\text{cprim}} := \text{PostAnalyzeCipherPolys}(\mathcal{V}_{\text{cprim}}, \mathcal{M}_{\text{fdh}})$   $\triangleright$  Retroactively verify assumptions

18:   $\mathcal{M}_G := \text{AnalyzeGroupMap}(\mathcal{M}_G, \mathcal{M}_K, \mathcal{V}_{\text{cprim}}, \mathcal{V}_{\text{cv}}, \mathcal{V}_{\text{knlr}}, \mathcal{V}_{\text{cnlr}}, \mathcal{R}_{\text{emat}})$ 

19:   $\mathcal{V}_{\text{evec}} := \text{AnalyzeSingles}(\mathcal{M}_K, \mathcal{R}_{\text{evec}})$ 
20:   $\mathcal{V}_{\text{emat}} := \text{AnalyzePairs}(\mathcal{M}_K, \mathcal{M}_G, \mathcal{R}_{\text{emat}})$ 
21:  return ( $v, \mathcal{V}_i$  for all occurring  $i, \mathcal{M}_j$  for all occurring  $j$ )

```

by inspecting the base sets of all quantifications of key and cipher polynomials. The maps $\mathcal{M}_K$ and $\mathcal{M}_G$ store the *kind* and *group* of every object and polynomial. They are threaded through the entire analysis and enforce consistency and uniqueness. The master-key and common variables are easy to analyze since they are given in separate JSON fields.

Analyzing the Key and Ciphertext Polynomials. Next, the key polynomials are analyzed. Each polynomial is kept track of in $\mathcal{M}_K$ and $\mathcal{M}_G$. Additionally, we fill the sets of key lone random and key non-lone random variables, checking for inconsistency along the way. Note that this analysis is “incomplete”, as the FDH mapping has not been analyzed yet. We assume that no variable is hashed and correct this assumption retroactively if needed.

The cipher polynomials are more difficult to analyze, as there are three types: *primary cipher polys* $\mathbf{c}$, *secondary cipher polys* $\mathbf{c}'$ and the *blinding poly* c_M . Primary cipher polynomials are all those mapped to either $\mathbb{G}$ or $\mathbb{H}$. In contrast, secondary cipher polynomials are mapped to $\mathbb{G}_T$. The blinding polynomial is the *unique* polynomial named $\mathbf{cm}$ which is also mapped to $\mathbb{G}_T$. Once separated, the cipher polynomials are analyzed analogously to the key polynomials.

Analyzing the FDH Mappings. At this stage, we can analyze the FDH mappings. Note that this was not possible earlier, because we need to “know” all objects to ensure that only common variables or key non-lone random variables are denoted to be hashed. We also make extensive use of our notion of equivalence (cf. Definition 11) to analyze the FDH mappings in combination with the group assignment. A detailed example can be found

in [Appendix C](#).

Re-Evaluating the Key and Ciphertext Polynomials. Based on a verified FDH assignment, the key and (primary) cipher polynomials are re-examined. This includes additional correctness checks involving the FDH mapping as well as partitioning the terms into *plain* and *hashed* terms. The latter is required because the code to be generated differs and plain terms have to be computed before hashed ones.

Analyzing the Group Assignment. The group assignment map $\mathcal{M}_G$ contains all explicit groups maps from all polynomials. However, this is not sufficient. Key and cipher non-lone random variables also need to be assigned to a group. This group cannot be inferred from the polynomials directly. Instead, the decryption pairings implicitly dictate that each non-lone variable is placed in the “opposite” group of the polynomial it is paired with. Further, a common variable b is placed in the same group as the cipher polynomials that contain b . This is required, because encryption only has access to g^b and not b .

Analyzing Decryption and Finalizing Analysis. Ultimately, the decryption vector $\mathbf{e}$ and matrices $\mathbf{E}$ and $\bar{\mathbf{E}}$ are analyzed, relying on all the information gathered in the previous steps. The final result is a huge object representing a *consistent* and *valid* ABE scheme.

4.4 Code Generation

We discuss our code generation module in more detail. Most of it is straightforward and follows known techniques from compiler engineering. The most notable exception is the handling of the concepts in our customized formal model and type system, including Quants and Idxs. Roughly, the quantifications correspond to for-loops, but the use of functions and indices that consume names produced by the quantifications complicate code generation, requiring associative data structures like hash maps.

Generating Dictionary Keys. To keep our tool agnostic from specific implementations, we assume a simple dictionary data structure that is indexed by strings. The challenge for our tool is then to generate code that *builds* the correct dictionary keys as strings. Our models of Vars (and by extension Polys), Quants and Idxs have been devised with precisely this goal in mind. The high-level idea is to build a dictionary key by concatenating small strings representing a Var and its Idxs into a larger string. Using the notion of similarity, we can ensure that this larger string uniquely represents the variable and that no collisions are created with variables that are not in the same equivalence class.

Running Example. We illustrate our approach to generating dictionary keys with an example using C++ style syntax, because C++ is strongly typed. Consider the variable

$$v = (r, [i_1 = (2, \text{id}), i_2 = (j, \text{ToLbL})], [q_1 = (j, \text{PosLssRows}, \text{LssRowToAttr})])$$

which features both a global and local map, and constant and quantified Idxs. [Listing 1](#) shows the corresponding code.

```

1  for (LSSRow j_global : env.get_pos_lsss_rows()) {
2      Attr j = env.ls_row_to_attr(j_global);
3      ...
4      idx = "";
5      idx += "r";
6      idx += "_{";
7      idx += "2";
8      idx += ",";
9      Lbl j_local_0 = env.attr_to_lbl(j);

```

```

10     idx += env.lbl_to_str(j_local_0);
11     idx += "}";
12     ...
13 }

```

Listing 1: Example of generated C++ code to build a string used as the index in an associative container for a quantified Var with a global and local map.

High-Level Structure: the For-Loop. The base set `PosLSSSRows` in the quantification is what the backend needs to iterate over. We use the string `env.get_pos_lsss_rows()` for this. The elements in this base set are of some specified QType called `LSSSRow`. We use the string `LSSSRow j_global` to represent these, where we use `j_global` to indicate that the quantification has a global map other than `id`. Further, it is typed to support exports for typed languages such as C++ (cf. [Appendix C](#)). Note that this would not have been possible if we had not installed the type system shown in [Section 3](#).

Evaluating the Maps. We now explain how the maps in the quantification and indices are evaluated. Because the indices are inferred from the quantification, we first evaluate the global map, `LSSSRowToAttr`. This maps `j_global` to an attribute of QType `Attr`, and thus, we store the result in `Attr j`. We use the same name `j` as in the quantification in our example, because consistency in naming facilitates debugging and cross referencing. Later, the local map is evaluated using a similar approach, but we use the inferred type for the indices to concretize the function. For example, we use the function `attr_to_lbl`, while the associated `IMap`, `ToLbl`, supports multiple domains. We do this, because we do not want to assume that the backends has function overloading.

Domain Separation. The generated code intentionally includes additional characters such as `_`, `{` and `,` to provide domain separation among indices. In the resolution of local maps, we also use domain separation in the code, as the same names may occur in the indices. For example, we append a `_0` to the variable name `j_local` to avoid issues if the same variable is reused again in the same loop.

4.5 Backend Design

After analyzing and verifying the ABE scheme given as input, our tool generates five output files as shown in [Figure 2](#). These files are designed to be integrated into runnable *backends*. We aim to impose as few restrictions as possible on the backends regarding the interaction with our output. [Figure 3](#) illustrates a generic backend. The five red components correspond directly to the five outputs of `pracy`. The `meta.json` file simply contains (optional) parameters or configuration options chosen by the backend developer. The remaining four outputs are plain text files, one for Setup, KeyGen, Encrypt and Decrypt, respectively.

Each code file is formatted according to the syntax specified by the export layer. The code itself is only a sequence of statements and is intended to be inserted *into* other code structures such as functions, classes or modules. One can, for example, do this via copy-pasting or (ab)using `#include` or `exec/eval` primitives. The central advantage we gain is flexibility. A function in the backend which hosts our generated code is free to have any desired name or signature. Further, it can contain custom code before and after ours. This is useful for declaring variables in statically typed languages, starting timers for benchmarks or performing any other operation. In [Figure 3](#), we show this with green boxes surrounding the generated code (red box). Our code is agnostic towards the way that the

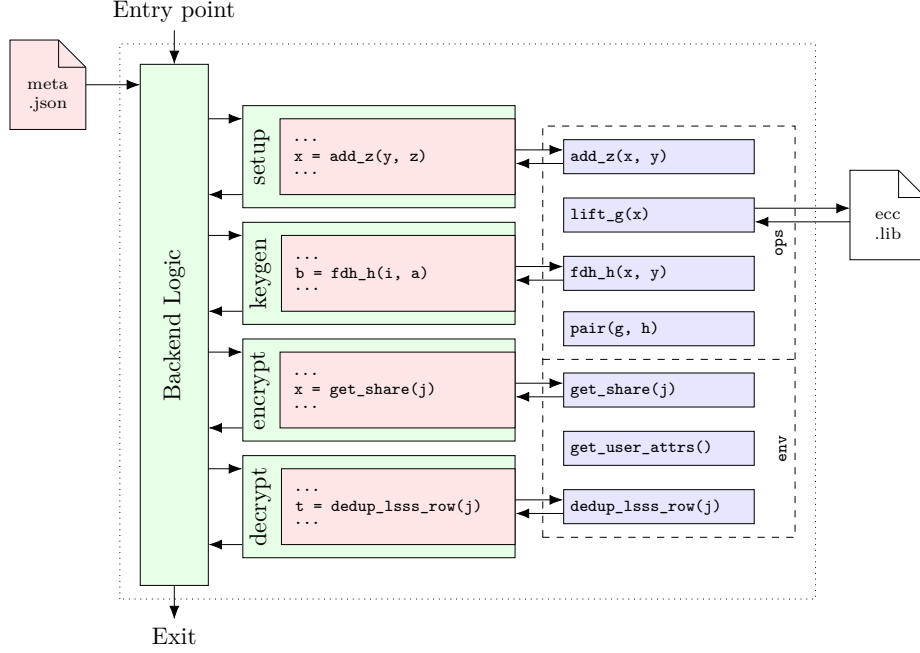


Figure 3: Schematic outline of our backend design. The areas in red are generated by our tool and correspond to the red output in Figure 2. Green highlights *arbitrary* logic, whereas blue indicates the API as specified in the export layer in our tool.

backend logic interacts with the host functions. Thus, the backend could serve any purpose, e.g., benchmarking, testing, or even a full-fledged server using ABE in deployment.

The only requirements we impose on the backend is to support all of our 39 IR statements. Most of them are translated to function calls. Each statement has a strictly defined “signature” and expected behavior, as detailed in Appendix D. Again, how these functions are implemented is up to the backend. We found it useful to partition the functions into two groups called *operations* and *environment* (*ops* and *env* in Figure 3). The former bundles all group operations over $\mathbb{Z}_p$, $\mathbb{G}$, $\mathbb{H}$ and $\mathbb{G}_T$. These could, for instance, be delegated to a third-party library for pairing-based cryptography. The environment, on the other hand, manages the access to predicate-specific components like attribute sets, secret sharing or the deduplication mappings.

4.5.1 Charm Backend

The first backend we implement is based on Charm [AGR12] and Python. Charm is a framework for rapid prototyping of cryptographic implementations and features – among other things – the implementation of commonly used pairing-friendly curve operations as well as a policy parser and associated secret sharing scheme.

However, Charm works internally with a different representation for the policies, using access trees [GPSW06a] and Shamir’s secret sharing scheme [Sha79]. To benefit from the user-friendly policy parser provided by Charm, we built an interface that translates the Charm data structures to hand-crafted data structures that interact more readily with the LSSS representation that we use in our JSON inputs and compiler, leveraging the conversions in [LCW10, dIPVA22b]. This interface also illustrates that using LSSS notation in our JSON input and compiler is not restrictive. Rather, it can capture essentially any policy representation used in ABE literature.

Our backend implementation also ensures that we can support, where needed, negations

in the policies and a more fine-grained representation of the attributes, e.g., attributes consisting of an authority, label and value, as well as less fine-grained representations, e.g., values only. The `meta.json` is in this case used to specify the attribute representation and the operators required by the scheme. In this way, the policies and sets defined in the tests can describe only those data that are needed. We also use the JSON file to specify whether a scheme supports small or large universes. Note that this configuration determines what inputs all the implemented algorithms receive.

The main purpose of this backend is to test the correctness of schemes implemented using `pracy`. To this end, it tests whether decryption succeeds and fails as expected for hand-crafted test cases. It additionally checks whether variables and polynomials in $\mathbb{G}$ and $\mathbb{H}$ are handled separately, and in the right groups. The backend also serves as an example of how a backend can be structured in such a way that schemes with many different properties can be captured, and how the `meta.json` can be utilized to achieve that.

4.5.2 RELIC Backend

The second backend we implement is based on RELIC [AGM⁺] and is written in C++. RELIC is a C library for pairing-based cryptography. In contrast to Charm, it is much more up to date regarding pairing-friendly curves, supporting various curves that provide 128, 192 or 256 bits of security, as well as highly optimized algorithms for group and pairing operations. We use C++ to leverage the standard library and have associative containers to model keys and ciphertexts in the large-universe setting.

The purpose of this backend is threefold. First, C++ is a different and more lower-level language than Python. Supporting a different language demonstrates the flexibility of our IR and export mechanism. We can, for instance, cope with up-front declarations of variables required by C++ which are particularly tricky with (multiple) local maps. Also, by generating code that compiles successfully, we implicitly test the correctness of our internal type tracking and propagation.

Second, we illustrate how different backends and their usage of `meta.json` can be. Whereas Charm provides full support for policy parsing and secret-sharing, this backend only contains the bare minimum to run correct benchmarks. Only conjunctions are supported as policies and the parsing as well as secret-sharing logic are limited in their functionality. Each attribute and literal is always modeled as an authority-label-value triple. Schemes which do not use labels or multiple authorities simply never access these values, since `pracy` does not generate the corresponding IR statements. Even the deduplication mapping τ is implemented as a function which constantly returns 1. This is valid, because the auto-generated policies contain pairwise unique literals and a proper implementation would equally evaluate to 1 in all cases. Consequently, the `meta.json` is *empty* for our RELIC backend. The little information required is passed via three compile-time parameters.

Lastly, we implement simple benchmarks to measure the performance of our schemes. To obtain results comparable with existing literature we mirror their choice of language and technology, i.e., C++ and RELIC. The benchmarks are evaluated in Section 5.2.3.

4.5.3 Concrete Suggestions for Backends

Although our two provided backends already give fully functional examples, we give some more suggestions on how to construct backends. As mentioned, the backend can be implemented in a target language for a larger ecosystem, e.g., Cloudflare’s CIRCL [FHK19] is implemented in Go and arkworks in Rust. The backend can also be designed to satisfy the requirements posed by the application in which the ABE implementations are used. For example, we may require a user-friendly API for the policies and attribute sets for practical applications such as Portunus [LVV⁺23]. Alternatively, we may want to optimize

and fairly benchmark multiple schemes [dIPVA22a], in which case we do not need a fully functional and user-friendly API for the policies. Additionally, for a good user experience, the API for the key generation, encryption and decryption algorithms may need to be more descriptive than the more generic descriptions in the compiler (Definition 4).

Parsing and Handling Properties. While such aspects are relatively straightforward to take into account when all schemes that need to be captured by the backend enjoy the same properties, it is more complex when various different properties are at play. Such properties include, e.g.,

- the ABE type, i.e., key-policy or ciphertext-policy;
- the support for small or large universes;
- the attribute representation, e.g., values only or authority-label-value triples
- the expressivity of the policies, e.g., ANDs, ORs, NOTs, thresholds;
- the bounds on any system parameters, e.g., the attribute sets or policy lengths;
- the support for multiple authorities.

These are typically considered part of the “environment”, albeit implicitly, and are therefore handled in the backend rather than in the compiler itself.

To properly capture behavior in the backend based on these properties, two approaches can be considered. The first option is to use the `meta.json` to pass along these properties to the backend, in which case we need to parse and handle the JSON file. This option has the advantage that the `meta.json` can be carefully constructed to include all options that are supported by the backend. However, it requires a lot of extra user input. The second option is to automatically derive, from the JSON input, which properties are satisfied by a scheme. For example, if the variables generated in the setup are independent of the attribute universe, then it supports large universes. Although this has the advantage that it requires less or no user input, such automated derivations are heuristic and may not be conclusive in all cases. For example, consider the dual-policy schemes in [AI09, AY15] or the decentralized KP-ABE scheme in [RVV24]. The first two are both key and ciphertext-policy based, whereas the second is a KP-ABE scheme and uses techniques from CP-ABE to tie keys generated by different authorities together. Although the latter should not be considered a dual-policy scheme, it has all the characteristics that we could use to assign the predicate “dual policy” for schemes of the first form.

Future Work: Using Automation to Optimize User Experience. In future work, it would be interesting to investigate which properties can be derived unequivocally from the scheme specification and for which we require the `meta.json` file. Furthermore, for such more general parse-and-handle approaches, we may not require the exported scheme files (in the target language), but it may be sufficient to consider the intermediate representation. It could be useful to construct a more generic property-deriving functionality in the compiler, which parses and handles the aforementioned properties on a more general level before passing along the extracted information (e.g., in the `meta.json`) to the backend.

5 Demonstrating and Evaluating pracy

In this section, we demonstrate the use of `pracy`, and we evaluate our new tool in various aspects. We do this to ascertain the quality of the generated code, illustrating that it is reasonably close to manually created code. To this end, we use existing ABE schemes from literature as test cases. We draw comparisons with existing implementations and debate correctness, timing performance as well as potential shortcomings.

5.1 Schemes Implemented

We implement five different schemes from literature. For now, we exclusively consider ciphertext-policy ABE schemes as we deem this variant more practically relevant for our prototype than the key-policy counterpart. The first three schemes are “classics” and have been included in various implementation efforts, such as ABE Squared [dIPVA22a]: the schemes by Waters (Wat11) [Wat11, §3], Rouselakis and Waters (RW13) [RW13], and Agrawal and Chase (AC17) [AC17]. Via ABE Squared, we obtain consistent performance metrics from a single source, which serve as a reference point.

The other two schemes have – to the best of our knowledge – not been implemented before, and demonstrate both the power of the Ven23 and ISABELLA compilers, as well as the coverage of `pracy`. The first is Construction 4.5 (C4.5) in [Ven23a], which is a decentralized large-universe scheme based on FDHs. It can be seen as a decentralized (and large-universe) variant of AC17. The second is Definition 16 (D16) of [Ven23b], which is also a decentralized large-universe scheme, but it additionally supports OT-type negations. It is among the most powerful schemes that our compiler can support. Note that both C4.5 and D16 feature an optimization based on randomness-reuse in the form of the deduplication mapping τ , like AC17.

Variations. For each scheme, we additionally create seven to eleven variations. These variations include

- moving key and ciphertext components between $\mathbb{G}$ and $\mathbb{H}$,
- marking more or less variables to be generated by an FDH,
- removing the deduplication mappings τ ,
- removing support for multiple authorities and
- adapting the coefficients in the key and ciphertext polynomials.

Not all variations are applicable to all five schemes, e.g., not all schemes use the deduplication mapping. As a result, we obtain 44 different scheme specifications for our testing purposes. The purpose of these variations is threefold. First, it showcases how easily minor changes can be applied to a specification. In our JSON input files, the above changes require only a few characters or lines to be adjusted. Second, the reference benchmarks from ABE Squared [dIPVA22a] use different group mappings to optimize either key generation, encryption or decryption. To establish a fair comparison, we employ the same group distributions. Third, we obtain more test cases to ensure correctness of our tool. This last point is also the reason why we include variations that weaken the expressiveness or increase the runtime. You can find a detailed listing of all implemented schemes and their variations in [Appendix E](#).

5.2 Performance Evaluation

Since our tool is the first of its kind, there is no related work to contrast itself with directly. For each of the 44 schemes, `pracy` parses, analyses and generates code in less than two seconds. This is practical for all intents and purposes and outperforms any manual work by far. We would like to point out explicitly, that such low execution times are ideally suited for design space explorations (DSEs) in future work.

To compare our tool meaningfully with existing work, we evaluate the performance of the generated code and compare it to ABE Squared. Such comparisons are delicate and only indirectly reflect our tool. We consider three categories for our comparison: code size, code readability and modularity, and execution speed. The former two only depend on

`pracy`, and the latter additionally depends the backend being used. Furthermore, although the code readability and modularity are inherently subjective, we do believe that code reusability is vital for effective and viable research.

5.2.1 Size of Generated Code

We begin by analyzing the lines of code emitted by `pracy` for each scheme and algorithm. The idea is to estimate how much our tool reduces the work required to obtain new implementations. To this end, we contrast the length of the input, i.e., a PES-Ven23 definition in JSON, to the length of the resulting code. In order to make the argument sound, we additionally compare our code with reference implementations.

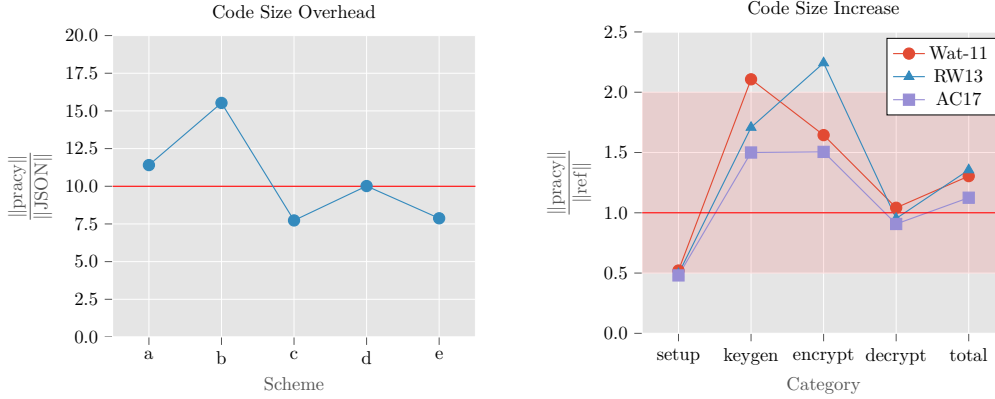
In Table 5, we show the line counts for all inputs and outputs. For the Wat11, RW13 and AC17 schemes (named `c`, `d` and `e` respectively) we additionally report the number of lines in their implementation in ABE Squared [dlPVA22a]. The numbers are taken after formatting the reference code to contain no blank lines or comments, and exactly one statement per line. Note that we summarize all variations of one scheme by giving ranges instead of concrete numbers. In general, the length of the code varies little within one scheme and lines of code is a delicate metric anyway.

Table 5: Lines of code generated by `pracy` compared to the reference implementations from ABE Squared [dlPVA22a]. The top row in each section shows the output length of our tool. Note that for C4.5 and D16 no reference implementation is available.

Scheme	Length JSON	Length Setup	Length Keygen	Length Encrypt	Length Decrypt	Total
a	33-36	29-40	80-89	157-167	109-113	379-408
	-	-	-	-	-	-
b	39-44	11-59	95-124	273-296	194-206	604-685
	-	-	-	-	-	-
c	27-29	14-23	55-63	74	61-64	209-224
	-	32-39	24-32	42-48	56-64	159-173
d	31-36	8-34	89-104	120	89-92	321-350
	-	42-43	51-62	52-55	93-97	239-256
e	27-29	14-23	58-65	72-74	62-65	214-227
	-	31-46	29-53	43-54	65-75	178-214

Comparison of JSON Input and Generated Code. First, we contrast the size of the generated code with the JSON specification of each scheme. This serves as an estimate of the work saved by using our compiler compared to manual implementations. Figure 4a shows the ratio between the two for all schemes. Clearly, the emitted code is at least 7.5 times longer than the original input JSON file. This fact indicates that our tool drastically reduces the work required to obtain new implementations of ABE schemes. For schemes C4.5 and D16 (a and b), the ratio between input and output length even exceeds 11 and 15, respectively. This is due to these schemes being more expressive and complex, e.g., supporting multiple authorities and, in the case of D16, OT-type negations. As a consequence, the implementation becomes significantly longer as additional key and ciphertext components as well as pairings have to be computed. The PES-Ven23 definition and thus the JSON specifications, on the other hand, only grows slightly. For instance, one additional key component results in an overhead of only two lines: one for the key polynomial and one for the new pairing.

Comparison of Our Generated Code with Manual Implementations. Second, we juxtapose the size of the generated code with reference implementations. This is not straightforward as each scheme in ABE Squared is implemented as a single function. We use the existing comments as delimiters between the four algorithms, but leave all other interleaved computations and instrumentalizations in place. Figure 4b illustrates the ratio between our generated code and the reference for the Wat11, RW13 and AC17 schemes. Roughly, we are within a factor of 2 regarding code size in all cases. For setup, `pracy`’s code is significantly smaller. This is because ABE Squared performs precomputations during setup to accelerate future group operations. These are reused in later algorithms leading to faster execution times (cf. Section 5.2.3). For key generation and encryption, our tools emits longer code because of our index building. By constructing the indices dynamically as strings, we stay closer to real-world use cases and maintain a closer relation to the JSON input. The latter leads to more accessible and easier to debug code. Interestingly, our index building does not affect the relative size of the code for decryption as much. Here, ABE Squared performs additional operations to benefit from shared-argument pairings or use optimized operations such as multi-base exponentiations [Möl01].



(a) Ratio between the JSON specification and the auto-generated implementation for all implemented schemes.

(b) Ratio between the auto-generated code and handcrafted (optimized) implementations from literature.

Figure 4: Relative size of our generated code compared to the input and reference code.

5.2.2 Readability and Modularity of Generated Code

Following the code size, we briefly investigate the code readability and modularity. Although this metric is intrinsically subjective, we put explicit efforts into creating accessible and modular code as well as retaining cross references to the input. Clear code is a cornerstone of our manual correctness analysis and vital for a prototype implementation, as there are no reference tools available.

The modularity rises naturally from the design of our backend infrastructure. Each algorithm is written to a different file and interacts with a clearly defined application programming interface (API) (cf. Appendix D). Precomputations, instrumentalizations for benchmarking, error handling and input-output behavior must be handled by the backend itself. Consequently, the generated code is purely related to the “logic” of the scheme.

The readability, on the other hand, is rooted in the way we handle variables names and index building as illustrated in Section 4.2 (Code Generation). In this step, we retain the (arbitrarily chosen) names which makes cross referencing trivial and natural.

5.2.3 Comparison of Benchmarks with ABE Squared

In order to further characterize the quality of the code emitted by `pracy`, we replicate the benchmarks done in [dIPVA22a] and compare the results. To this end, we use our RELIC backend, as ABE Squared uses the same library. We pick BLS12-381 [BLS04] as a commonly used pairing-friendly curve and construct individual variants of Wat11, RW13 and AC17 to match the OE, OK and OD instantiations. We further obtain the small and large-universe variants by marking the corresponding variables to be computed via FDHs if necessary. A detailed listing can be found in Appendix E. We run the benchmarks on Ubuntu 22.04 with 16GB of RAM and Intel Core i7 (12th generation). We apply the same configuration options for RELIC and use the cycle counts from Table 9 [dIPVA22a].

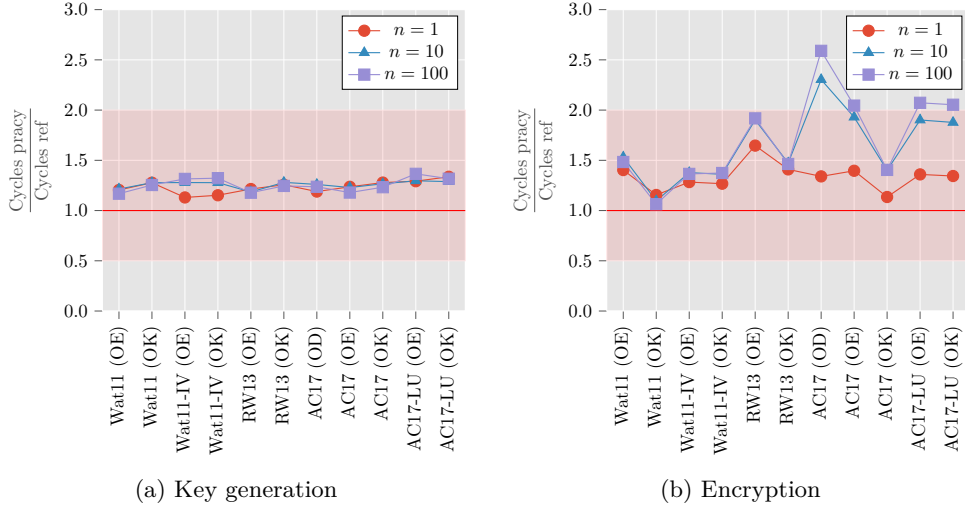


Figure 5: Relative execution times for key generation and encryption (with varying n) of `pracy` using our RELIC backend compared with ABE Squared.

Evaluating Our Results. We are primarily interested in *relative* execution times, as these better reflect the implementation and not the structure of the scheme itself. We omit the setup algorithm as it is only run once and its performance is generally not considered essential for ABE implementations. Figure 5 shows the relative performance for key generation and encryption. For key generation, our tool’s runtime is consistently at most 1.5 times as long as the hand-optimized reference for all set sizes. This is consistent with the fact that key generation is simple in all cases, and the main distinction between our code and ABE Squared is likely the use of precomputations in ABE Squared.

For encryption, our implementations are at most twice as slow in most cases. The main reason is that ABE Squared uses precomputations to accelerate encryption. Our backend, however, does not use any such optimizations. Additionally, the secret sharing is included in the timings, which could cause some overhead. Whereas ABE Squared relies on Shamir’s secret sharing scheme (from OpenABE [WGW⁺]), our backend uses a custom implementation based on LSSS matrices and the secret sharing scheme in [LW10].

Large Discrepancies for AC17 Encryption. The exceptions are in the AC17 scheme and its large-universe variant, which is much worse than twice as slow for policies of size 10 and 100. The cause for this is the deduplication mapping τ used in AC17. While we correctly compute the mapping and only *use* one variable s_1 in the polynomials, we sample a non-lone random s_j (and exponentiate) for each attribute (and then discard it). This

is caused by how the code is generated, i.e., the for-loops are always over the domain of the global map associated with indices (via the quantifications). To improve this, we would need to be able to create for-loops over the co-domain of the local map and reverse all function applications in the code generation. On a compiler level, there is no way of knowing which of the two is better (without explicitly modeling this). We leave such improvements for future work.

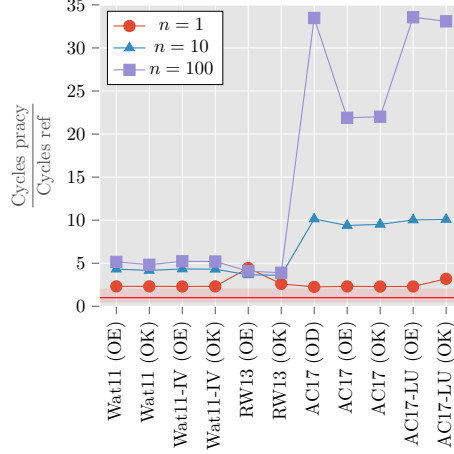


Figure 6: Relative execution times for decryption (with varying n) of `pracy` using our RELIC backend compared with ABE Squared.

Evaluating Decryption. For the decryption, our RELIC backend is about 2 to 6 times slower for Wat11, Wat11-IV and RW13. The cause for the relative slowdown is the lack of optimization strategies (e.g., multi-pairings and grouping of pairing products sharing the same argument). In this case, the effects are worse, since such optimizations are much more effective compared to optimization through precomputations. For the AC17 scheme, the effect is even worse, as it makes use of deduplication mappings to reduce the pairing costs close to a small constant. Since our backend does not apply this optimization, the number of pairings is significantly larger, resulting in slowdown factors of over 30.

Summarizing Remarks on Execution Times. In summary, the automatically generated code emitted by `pracy` is mostly competitive with the handcrafted reference implementation. With the exception of encryption and decryption of the AC17 scheme, the slowdown factor is between 1 and 6, while using less optimized algorithms. The benchmarks also suggest that our tool could ultimately benefit greatly from internal support for optimization.

5.2.4 New Benchmarks for Decentralized Schemes

We additionally provide the first implementation in literature for two schemes: C4.5 and D16. Their performance characteristics are listed in Table 6 following the methodology used in [dlPVA22a]. That means, the policy is a conjunction of n pairwise unique attributes. Each attribute is further associated with a unique authority. Since D16 supports OT-type negations, it is listed twice. In the latter form, all attributes in the policy are negated. We use the same setup and configuration of RELIC as above to ensure the numbers are comparable with Table 9 in [dlPVA22a]. By comparing the cycle counts for key generation and decryption, one can clearly see the differences in speed for operations in $\mathbb{G}$ and $\mathbb{H}$ by contrasting the OE and OK variants. Also, decryption of policies with negations is more costly, as it involves more group operations.

Table 6: Cycle counts for first implementations of C4.5 and D16. The implementation uses BLS12-381 with RELIC. The policies of size n are conjunctions of n pairwise unique authorities and attributes. The cycle counts are given in kilocycles.

Algorithm	n	Constr. 4.5		Def. 16		Def. 16 (neg)	
		OE	OK	OE	OK	OE	OK
Setup	1	809	965	857	1081	822	1051
	10	8785	10134	9122	10990	9157	10974
	100	88682	106901	93645	111811	92915	107935
KeyGen	1	1666	1265	2232	2003	2337	1882
	10	18591	12999	24900	19941	24396	19424
	100	181577	133180	246657	192188	241643	187175
Encrypt	1	3033	3343	3788	4390	4350	4799
	10	28708	32066	34623	39501	40730	45070
	100	275286	323158	341234	392162	393608	437419
Decrypt	1	5903	5753	6526	6750	9512	9642
	10	68833	60639	69042	66690	98923	100097
	100	625997	612077	649869	633221	936291	913633

5.3 Correctness of the Tool

Another major evaluation criterion for `pracy` is its correctness. To systematically and thoroughly address the correctness of `pracy`, we consider three different aspects:

- Correct analysis and processing of valid inputs.
- Correct rejection of invalid or malformed inputs.
- Correct generation of code.

For each of the above correctness aspects, we employ different techniques and methodologies. In total, our implementation consists of approximately 6000 lines of code and 392 test cases spanning over 26000 lines of testing code.

Correct Analysis of Inputs. To ensure that valid inputs are accepted and parsed as intended, we use conventional unit and integration tests. We also rely on open-source libraries to avoid rewriting software and benefit from their maturity. Although it slightly decreases usability, we err on the side of caution. As detailed in [Section 3.2](#), our correctness checks disallow ambiguities and thus risk false negatives.

Correct Rejection of Invalid or Malformed Inputs. Detecting malformed inputs, on the other hand, is crucial for `pracy`. Failure to do so would lead to a user running or benchmarking insecure schemes without notice. Thus, we care very much about *minimizing* false positives, i.e., invalid inputs that are not detected as such. To facilitate asserting the correctness of our implementation, the error conditions we test are integrated into the design of the tool itself. We have over 102 error conditions and model each as an individual exception object. This allows us to track which conditions are tested, by simply listing all custom exception types. Further, we can find where each condition is tested and if at least one unit test covers this condition using simple text-based search tools. [Table 7](#) lists some error conditions in combination with the objects for which we check them. A check mark indicates that an error condition is applicable and tested for the corresponding object. Note that many more intricate scenarios involving combinations of multiple objects are also tested, but not shown in the table.

Table 7: Excerpt of our 102 correctness criteria to detect and reject malformed inputs. A check mark (✓) indicates that the corresponding condition is tested for the objects. We use the abbreviations [P]olynomials, [L]one [R]andom, [N]on-[L]one [R]andom and [S]pecial [L]one [R]andom variables.

Condition	MSKs	CVs	Key Polys			Cipher Polys				Decr.		FDH Map
			Ps	LRs	NLRs	Ps	LRs	NLRs	SLRs	Vec	Mat	
Non-emptiness	✓	✓	✓			✓					✓	
Uniqueness	✓	✓	✓			✓						
Type error	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Illegal quantifications	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Unused quantifications	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Illegal special variable	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Illegal variable kind	✓	✓	✓	✓	✓	✓	✓	✓	✓			
Uncomputable term			✓			✓						
Invalid unary term			✓			✓						
Invalid binary term			✓			✓						
Invalid term			✓			✓						
Invalid expression			✓			✓				✓	✓	
Invalid group			✓			✓						
Conflicting variable kind	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	

Correct Generation of Code. Lastly, we consider the correctness of the generated code. To this end, we have extensive integration tests for *all* implemented schemes and variants explicitly asserting the correctness of the entire generated code. This is the main contributing factor to our over 26000 lines of testing code. Such a extensive test coverage is essential, but can become difficult to oversee. In our case, we can exploit the similarity among scheme variants and the systematic code generation. Thus, the tool `diff` can be used to further verify the (minor) differences. Additionally, we have two independently developed backends based on RELIC and Charm, respectively. Each of these features runtime tests to further assert correctness.

6 Future Work and Conclusion

6.1 Future Work

As mentioned, our work aims to provide the foundations for future work in the field of pairing-based ABE. We highlight some potential design space explorations below.

Automated Optimization. A natural extension building on `pracy` would be to automate optimization efforts. To this end, we have already included placeholder layers (Optimization I and II). Currently, our implementations do not exploit many optimization tricks that are usually applied in the manual implementation of schemes [dIPVA22a], such as the grouping of pairing products that share the same argument, the sharing of the final exponentiation in a product of pairings or the use of fixed-base exponentiations. Furthermore, the user needs to manually determine in which groups the pair encodings are instantiated, but this can, in theory, be done automatically, using similar approaches as in [AGH13, AGH15]. An additional optimization that can be done automatically is the target-group-to-source-group conversion that the ISABELLA compiler supports under some conditions [RVV24].

Automated Security Analysis. Another interesting pathway to explore is how our formal model and type system as well as our parser can be used in automated security analysis. Where the parsers in [ABGW17, dIPVA23, RVV24] could only consider the security of fixed predicates, our parser retains the interaction between the predicates and the pair encodings.

This could improve automated security analyses considerably, e.g., by automatically analyzing the schemes with different choices of predicates, or something more involved.

Automated Transformations. An additional feature that we could add in future work is the automated support for transformations on pair encodings that are proven to preserve their security [ABS16, Att19, AT20, Amb21, VB24]. For example, theoretical compilers only achieve security against chosen-plaintext attacks. To resist against chosen-ciphertext attacks, we could automate the application of the transformation in [VB24].

Improving Usability. To further improve the usability of our tool (and associated backends), we can try to reduce the required information in the JSON file even more. As already mentioned in Section 4.5.3, we can investigate which properties (that we may have to include as input to the `meta.json`) can be automatically derived from the pair encoding specification. Further, it would be interesting to investigate whether the decryption algorithm can be automatically derived from the key and ciphertexts. To this end, we require a more algebraic approach, similar to the ACABELLA prover [dlPVA23, RVV24].

Broadening the Scope. Another focus for future work could be to extend the scope of our formal model and the compiler. Currently, the compiler supports only ciphertext-policy schemes, despite the fact that the formal model can also capture key-policy variants [SW05, GPSW06a]. This is because our tool does not support the generation of code that shares master-key or common variables yet. Furthermore, we target Boolean formulas, because they are practically relevant, but it is possible that we can support more fine-grained predicates, e.g., regular languages [Wat12, AC17]. Additionally, we could extend the formal model and tool with split-predicate molds that are also part of the ISABELLA compiler [RVV24], which can be used to obtain other functional improvements to ABE beyond single and multi-authority ABE.

Lastly, although our scheme can cover many existing schemes for Boolean formulas already, some schemes require auxiliary mappings to the policies or sets [AHM⁺16, VA22, VA23]. These schemes, as well as many schemes supporting negations [OSW07, Att19, RVV24], also require “conditional quantification” in the definition of the polynomials, because these may contain sums over sets for which certain conditions hold. These are related to the predicate as well, but they require us to extend the formal model, as such conditions need to be captured in the “quantification” of the sum, and we do not do this.

6.2 Conclusion

In this work, we have presented `pracy`: a tool that, on input a PES, automatically implements an ABE scheme that is obtained with the Ven23 and ISABELLA compilers. To enable this, we have first devised the theoretical foundations needed to model the interaction between the predicates and the pair encodings. Our tool illustrates that our model and type system are compatible with automation. To make the tool a valuable foundation for future work, we have built it with modularity, extensibility and usability in mind. We have also demonstrated the use of the tool, and shown that the quality of its automatically generated implementations are not much worse than manually implemented schemes, although there is still room for optimization. Lastly, we consider the active maintenance and support vital for our tool to flourish and be a useful contribution to the community. We therefore encourage everyone to explore, use and contribute to `pracy`.

Acknowledgments. The authors would like to honor the memory of Antonio de la Piedra, who worked on the `pracy` compiler in its early stages. Sven Argo and Tim Güneysu have been supported by the Deutsche Forschungsgemeinschaft (DFG, German Research

Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972. Many thanks to Jan Richter-Brockmann as well as the anonymous reviewers for the valuable feedback and comments.

References

- [ABGW17] Miguel Ambrona, Gilles Barthe, Romain Gay, and Hoeteck Wee. Attribute-based encryption in the generic group model: Automated proofs and new constructions. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 647–664. ACM Press, October / November 2017.
- [ABS16] Miguel Ambrona, Gilles Barthe, and Benedikt Schmidt. Automated unbounded analysis of cryptographic constructions in the generic group model. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 822–851. Springer, Berlin, Heidelberg, May 2016.
- [ABS17] Miguel Ambrona, Gilles Barthe, and Benedikt Schmidt. Generic transformations of predicate encodings: Constructions and applications. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 36–66. Springer, Cham, August 2017.
- [AC16] Shashank Agrawal and Melissa Chase. A study of pair encodings: Predicate encryption in prime order groups. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A, Part II*, volume 9563 of *LNCS*, pages 259–288. Springer, Berlin, Heidelberg, January 2016.
- [AC17] Shashank Agrawal and Melissa Chase. Simplifying design and analysis of complex predicate encryption schemes. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part I*, volume 10210 of *LNCS*, pages 627–656. Springer, Cham, April / May 2017.
- [AGH13] Joseph A. Akinyele, Matthew Green, and Susan Hohenberger. Using SMT solvers to automate design tasks for encryption and signature schemes. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 399–410. ACM Press, November 2013.
- [AGH15] Joseph A. Akinyele, Christina Garman, and Susan Hohenberger. Automating fast and secure translations from type-I to type-III pairing schemes. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 1370–1381. ACM Press, October 2015.
- [AGHP12] Joseph A. Akinyele, Matthew Green, Susan Hohenberger, and Matthew W. Pagano. Machine-generated algorithms, proofs and software for the batch verification of digital signature schemes. In Ting Yu, George Danezis, and Virgil D. Gligor, editors, *ACM CCS 2012*, pages 474–487. ACM Press, October 2012.
- [AGM⁺] D. F. Aranha, C. P. L. Gouvêa, T. Markmann, R. S. Wahby, and K. Liao. RELIC is an Efficient Library for Cryptography. <https://github.com/relic-toolkit/relic>.
- [AGOT14] Masayuki Abe, Jens Groth, Miyako Ohkubo, and Takeya Tango. Converting cryptographic schemes from symmetric to asymmetric bilinear groups. In

- Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 241–260. Springer, Berlin, Heidelberg, August 2014.
- [AGR12] Joseph A. Akinyele, Matthew Green, and Aviel D. Rubin. Charm: A framework for rapidly prototyping cryptosystems. In *NDSS 2012*. The Internet Society, February 2012.
- [AHM⁺16] Nuttapong Attrapadung, Goichiro Hanaoka, Tsutomu Matsumoto, Tadanori Teruya, and Shota Yamada. Attribute based encryption with direct efficiency tradeoff. In Mark Manulis, Ahmad-Reza Sadeghi, and Steve Schneider, editors, *ACNS 16 International Conference on Applied Cryptography and Network Security*, volume 9696 of *LNCS*, pages 249–266. Springer, Cham, June 2016.
- [AHO16] Masayuki Abe, Fumitaka Hoshino, and Miyako Ohkubo. Design in type-I, run in type-III: Fast and scalable bilinear-type conversion using integer programming. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part III*, volume 9816 of *LNCS*, pages 387–415. Springer, Berlin, Heidelberg, August 2016.
- [AI09] Nuttapong Attrapadung and Hideki Imai. Dual-policy attribute based encryption. In Michel Abdalla, David Pointcheval, Pierre-Alain Fouque, and Damien Vergnaud, editors, *ACNS 09 International Conference on Applied Cryptography and Network Security*, volume 5536 of *LNCS*, pages 168–185. Springer, Berlin, Heidelberg, June 2009.
- [Amb21] Miguel Ambrona. Generic negation of pair encodings. In Juan Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 120–146. Springer, Cham, May 2021.
- [AT20] Nuttapong Attrapadung and Junichi Tomida. Unbounded dynamic predicate compositions in ABE from standard assumptions. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 405–436. Springer, Cham, December 2020.
- [Att14] Nuttapong Attrapadung. Dual system encryption via doubly selective security: Framework, fully secure functional encryption for regular languages, and more. In Phong Q. Nguyen and Elisabeth Oswald, editors, *EUROCRYPT 2014*, volume 8441 of *LNCS*, pages 557–577. Springer, Berlin, Heidelberg, May 2014.
- [Att16] Nuttapong Attrapadung. Dual system encryption framework in prime-order groups via computational pair encodings. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 591–623. Springer, Berlin, Heidelberg, December 2016.
- [Att19] Nuttapong Attrapadung. Unbounded dynamic predicate compositions in attribute-based encryption. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 34–67. Springer, Cham, May 2019.
- [AVAG] Sven Argo, Marloes Venema, Adrian Ackermann, and Tim Güneysu. pracy: A Practical Compiler for Attribute-Based Encryption in Python. <https://github.com/lincolncryptools/pracy>.
- [AY15] Nuttapong Attrapadung and Shota Yamada. Duality in ABE: Converting attribute based encryption for dual predicate and dual policy via computational encodings. In Kaisa Nyberg, editor, *CT-RSA 2015*, volume 9048 of *LNCS*, pages 87–105. Springer, Cham, April 2015.

- [Bei96] Amos Beimel. *Secure Schemes for Secret Sharing and Key Distribution*. PhD thesis, Ben Gurion University, 1996.
- [BFF⁺14] Gilles Barthe, Edvard Fagerholm, Dario Fiore, John C. Mitchell, Andre Scedrov, and Benedikt Schmidt. Automated analysis of cryptographic assumptions in generic group models. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 95–112. Springer, Berlin, Heidelberg, August 2014.
- [BGS15] Gilles Barthe, Benjamin Grégoire, and Benedikt Schmidt. Automated proofs of pairing-based cryptography. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 1156–1168. ACM Press, October 2015.
- [BLS04] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. On the selection of pairing-friendly groups. In Mitsuru Matsui and Robert J. Zuccherato, editors, *SAC 2003*, volume 3006 of *LNCS*, pages 17–25. Springer, Berlin, Heidelberg, August 2004.
- [BSW07] John Bethencourt, Amit Sahai, and Brent Waters. Ciphertext-policy attribute-based encryption. In *2007 IEEE Symposium on Security and Privacy*, pages 321–334. IEEE Computer Society Press, May 2007.
- [CGW15] Jie Chen, Romain Gay, and Hoeteck Wee. Improved dual system ABE in prime-order groups via predicate encodings. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 595–624. Springer, Berlin, Heidelberg, April 2015.
- [Cha07] Melissa Chase. Multi-authority attribute based encryption. In Salil P. Vadhan, editor, *TCC 2007*, volume 4392 of *LNCS*, pages 515–534. Springer, Berlin, Heidelberg, February 2007.
- [dlPVA22a] Antonio de la Piedra, Marloes Venema, and Greg Alpár. ABE squared: Accurately benchmarking efficiency of attribute-based encryption. *IACR TCHES*, 2022(2):192–239, 2022.
- [dlPVA22b] Antonio de la Piedra, Marloes Venema, and Greg Alpár. ABE squared: Accurately benchmarking efficiency of attribute-based encryption. Cryptology ePrint Archive, Report 2022/038, 2022.
- [dlPVA23] Antonio de la Piedra, Marloes Venema, and Greg Alpár. ACABELLA: Automated (crypt)analysis of attribute-based encryption leveraging linear algebra. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 3269–3283. ACM Press, November 2023.
- [EHK⁺13] Alex Escala, Gottfried Herold, Eike Kiltz, Carla Ràfols, and Jorge Villar. An algebraic framework for Diffie-Hellman assumptions. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 129–147. Springer, Berlin, Heidelberg, August 2013.
- [ETS18] ETSI. ETSI TS 103 458 (V1.1.1). Technical specification, European Telecommunications Standards Institute (ETSI), 2018.
- [FEN] The FENTEC project. <https://github.com/fentec-project>.
- [FHK19] Armando Faz-Hernandez and Kris Kwiatkowski. *Introducing CIRCL: An Advanced Cryptographic Library*. Cloudflare, June 2019. Available at <https://github.com/cloudflare/circl>. v1.4.0 Accessed Aug, 2024.

- [GPSW06a] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 89–98. ACM Press, October / November 2006. Available as Cryptology ePrint Archive Report 2006/309.
- [GPSW06b] Vipul Goyal, Omkant Pandey, Amit Sahai, and Brent Waters. Attribute-based encryption for fine-grained access control of encrypted data. Cryptology ePrint Archive, Report 2006/309, 2006.
- [HV19] Susan Hohenberger and Satyanarayana Vusirikala. Are these pairing elements correct?: Automated verification and applications. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 923–939. ACM Press, November 2019.
- [KL10] Seny Kamara and Kristin Lauter. Cryptographic cloud storage. In Radu Sion, Reza Curtmola, Sven Dietrich, Aggelos Kiayias, Josep M. Miret, Kazuo Sako, and Francesc Sebé, editors, *FC 2010 Workshops*, volume 6054 of *LNCS*, pages 136–149. Springer, Berlin, Heidelberg, January 2010.
- [Lar] Lark - a parsing toolkit for Python. <https://github.com/lark-parser/lark>.
- [LCW10] Zhen Liu, Zhenfu Cao, and Duncan S. Wong. Efficient generation of linear secret sharing scheme matrices from threshold access trees. Cryptology ePrint Archive, Report 2010/374, 2010.
- [LVV⁺23] Watson Ladd, Tanya Verma, Marloes Venema, Armando Faz-Hernández, Brendan McMillan, Avani Wildani, and Nick Sullivan. Portunus: Re-imagining access control in distributed systems. In Julia Lawall and Dan Williams, editors, *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, pages 35–52. USENIX Association, 2023.
- [LW10] Allison Lewko and Brent Waters. Decentralizing attribute-based encryption. Cryptology ePrint Archive, Report 2010/351, 2010.
- [LW11a] Allison B. Lewko and Brent Waters. Decentralizing attribute-based encryption. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 568–588. Springer, Berlin, Heidelberg, May 2011.
- [LW11b] Allison B. Lewko and Brent Waters. Unbounded HIBE and attribute-based encryption. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 547–567. Springer, Berlin, Heidelberg, May 2011.
- [Möl01] Bodo Möller. Algorithms for multi-exponentiation. In Serge Vaudenay and Amr M. Youssef, editors, *SAC 2001*, volume 2259 of *LNCS*, pages 165–180. Springer, Berlin, Heidelberg, August 2001.
- [OSW07] Rafail Ostrovsky, Amit Sahai, and Brent Waters. Attribute-based encryption with non-monotonic access structures. In Peng Ning, Sabrina De Capitani di Vimercati, and Paul F. Syverson, editors, *ACM CCS 2007*, pages 195–203. ACM Press, October 2007.
- [OT10] Tatsuaki Okamoto and Katsuyuki Takashima. Fully secure functional encryption with general relations from the decisional linear assumption. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 191–208. Springer, Berlin, Heidelberg, August 2010.

- [RVV24] Doreen Riepel, Marloes Venema, and Tanya Verma. ISABELLA: Improving structures of attribute-based encryption leveraging linear algebra. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 4628–4642. ACM Press, October 2024.
- [RW13] Yannis Rouselakis and Brent Waters. Practical constructions and new proof methods for large universe attribute-based encryption. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 463–474. ACM Press, November 2013.
- [RW15] Yannis Rouselakis and Brent Waters. Efficient statically-secure large-universe multi-authority attribute-based encryption. In Rainer Böhme and Tatsuaki Okamoto, editors, *FC 2015*, volume 8975 of *LNCS*, pages 315–332. Springer, Berlin, Heidelberg, January 2015.
- [RW22] Doreen Riepel and Hoeteck Wee. FABEO: Fast attribute-based encryption with optimal security. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2491–2504. ACM Press, November 2022.
- [Sha79] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.
- [SRGS12] Nuno Santos, Rodrigo Rodrigues, Krishna P. Gummadi, and Stefan Saroiu. Policy-sealed data: A new abstraction for building trusted cloud services. In Tadayoshi Kohno, editor, *Proceedings of the 21th USENIX Security Symposium, Bellevue, WA, USA, August 8-10, 2012*, pages 175–188. USENIX Association, 2012.
- [SW05] Amit Sahai and Brent R. Waters. Fuzzy identity-based encryption. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 457–473. Springer, Berlin, Heidelberg, May 2005.
- [TKN20] Junichi Tomida, Yuto Kawahara, and Ryo Nishimaki. Fast, compact, and expressive attribute-based encryption. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part I*, volume 12110 of *LNCS*, pages 3–33. Springer, Cham, May 2020.
- [VA22] Marloes Venema and Greg Alpár. TinyABE: Unrestricted ciphertext-policy attribute-based encryption for embedded devices and low-quality networks. In Lejla Batina and Joan Daemen, editors, *AFRICACRYPT 22*, volume 2022 of *LNCS*, pages 103–129. Springer, Cham, July 2022.
- [VA23] Marloes Venema and Greg Alpár. GLUE: Generalizing unbounded attribute-based encryption for flexible efficiency trade-offs. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 652–682. Springer, Cham, May 2023.
- [VAH23] Marloes Venema, Greg Alpár, and Jaap-Henk Hoepman. Systematizing core properties of pairing-based attribute-based encryption to uncover remaining challenges in enforcing access control in practice. *DCC*, 91(1):165–220, 2023.
- [VB24] Marloes Venema and Leon Botros. Using predicate extension for predicate encryption to generically obtain chosen-ciphertext security and signatures. *CiC*, 1(1):6, 2024.

- [Ven23a] Marloes Venema. *Attaining Basically Everything in Attribute-Based Encryption. Simplifying the Design of Practical Schemes Via Pair Encodings*. PhD thesis, Radboud University, 2023.
- [Ven23b] Marloes Venema. A practical compiler for attribute-based encryption: New decentralized constructions and more. In Mike Rosulek, editor, *CT-RSA 2023*, volume 13871 of *LNCS*, pages 132–159. Springer, Cham, April 2023.
- [Wat11] Brent Waters. Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In Dario Catalano, Nelly Fazio, Rosario Gennaro, and Antonio Nicolosi, editors, *PKC 2011*, volume 6571 of *LNCS*, pages 53–70. Springer, Berlin, Heidelberg, March 2011.
- [Wat12] Brent Waters. Functional encryption for regular languages. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 218–235. Springer, Berlin, Heidelberg, August 2012.
- [Wee14] Hoeteck Wee. Dual system encryption via predicate encodings. In Yehuda Lindell, editor, *TCC 2014*, volume 8349 of *LNCS*, pages 616–637. Springer, Berlin, Heidelberg, February 2014.
- [WGW⁺] Brent Waters, Matthew Green, Susan Hohenberger Waters, J. Ayo Akinyele, Alan M. Dunn, and Michael Rushanan. The OpenABE Library. <https://github.com/zeutro/openabe>.
- [WMZV16] Frank Wang, James Mickens, Nickolai Zeldovich, and Vinod Vaikuntanathan. Sieve: Cryptographically enforced access control for user data in untrusted clouds. In Katerina J. Argyraki and Rebecca Isaacs, editors, *13th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2016, Santa Clara, CA, USA, March 16-18, 2016*, pages 611–626. USENIX Association, 2016.

A Security Definitions and Results

For completeness, we introduce the security definition omitted in [Section 2](#). We also summarize the security results from ISABELLA [\[RVV24\]](#).

A.1 Security against Chosen-Plaintext Attacks

We define the security game capturing security against chosen-plaintext attacks (CPA) as in [\[AC17\]](#). The game IND-CPA between a challenger and an adversary $\mathcal{A}$ proceeds as follows:

- **Setup phase:** The challenger runs $\text{Setup}(\lambda)$ to obtain MPK and MSK, and sends the master public key MPK to $\mathcal{A}$.
- **First query phase:** $\mathcal{A}$ queries secret keys for $y \in \mathcal{Y}$, and obtains $\text{USK}_y \leftarrow \text{KeyGen}(\text{MSK}, y)$ in response.
- **Challenge phase:** $\mathcal{A}$ specifies some $x^* \in \mathcal{X}$ such that for all y in the first key query phase, we have $P(x^*, y) = 0$, and sends y to the challenger. The challenger flips a coin, i.e., $\delta \in_R \{0, 1\}$, computes $(\text{CT}_{x^*}, K_0) \leftarrow \text{Encrypt}(\text{MPK}, x^*)$ and $K_1 \in_R \mathcal{K}$, and sends the ciphertext and key $(\text{CT}_{x^*}, K_\delta)$ to $\mathcal{A}$.
- **Second query phase:** This phase is identical to the first query phase, with the additional restriction that the adversary can only query $y \in \mathcal{Y}$ such that $P(x^*, y) = 0$.

- **Decision phase:** $\mathcal{A}$ outputs a guess δ' for δ .

Definition 13 (Security against CPA). Let Γ be an ABE scheme and let $\mathcal{A}$ be an adversary in the above game. We define the advantage of δ as

$$\text{Adv}_{\Gamma, \mathcal{A}}^{\text{IND-CPA}}(\lambda) = \left| \Pr[\delta' = \delta] - \frac{1}{2} \right|.$$

We say that Γ is fully secure if all polynomial-time adversaries $\mathcal{A}$ have at most a negligible advantage in this security game, i.e., $\text{Adv}_{\Gamma, \mathcal{A}}^{\text{IND-CPA}}(\lambda) \leq \text{negl}(\lambda)$.

A.2 Security of the ISABELLA Compiler

We summarize the security results proven in the ISABELLA compiler. Intuitively, these results are centered around the special selective symbolic property, which needs to hold for the given PES-ISA scheme to be compiled. Roughly, the special selective symbolic property (SSSP) states that some substitution vectors and matrices exist for the variables in the PES-ISA, such that, if the polynomials are evaluated on those vectors and matrices, they result in all-zero vectors. The only exception is the decapsulated key $e(g, h)^{c_M}$, for which we require that it is evaluated to a value that is non-zero. Additionally, for multi-authority schemes, we need security against corruptions. Therefore, this definition considers the notion of corruptable variables. These need to be substituted by all-zero vectors and matrices.

Once the SSSP is proven for some PES-ISA, the schemes compiled with the ISABELLA compiler are proven secure in two ways. The first security result ensures that fully adaptive security is proven, in the generic group model. The second security result ensures that static security is proven, under a q -type assumption. Both results rely on the random oracle model if the key generation can be queried in multiple stages.

B Approximate PES and Their Security

As mentioned in [Section 2.4](#), we introduce the notion of approximate pair encodings to enable more efficient instantiations of PES in practice. The most notable reason why we do this is to enable a more efficient and general implementation of the access structures. We define approximate PES in such a way that they can be readily compiled with the Ven23 and ISABELLA compilers. To prove security of the resulting scheme, we also propose a statistical security notion for approximate PES that captures that the scheme is statistically indistinguishable from some actual PES. (Therefore, the approximate PES sufficiently approximates and actual PES.) If the symbolic property holds for the actual PES, then the security of the compiled approximate PES is implied by the security of the compiled actual PES.

B.1 Definition of Approximate Pair Encodings

For simplicity, we formalize the notion of approximate pair encodings based on PES-Ven23 ([Definition 3](#)). Given that PES-ISA is a fairly straightforward extension of PES-Ven23, this adaptation can also be readily applied to PES-ISA. In the following definition, we highlight the additions compared to the original definition in [red](#).

Definition 14 (Approximate pair encodings). An approximate pair encoding scheme for pair encodings from the classes of schemes covered by the Ven23 and ISABELLA compilers, and for a predicate $P: \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ and prime number p , with optionally some additional parameters par , is given by four deterministic polynomial-time algorithms as described below.

- $\text{Param}(\text{par}) \rightarrow (n_\alpha, n_b, \alpha, \mathbf{b})$: On input parameters par , the algorithm outputs $n_\alpha, n_b \in \mathbb{N}$ that specify the number of master key variables and common variables, respectively, which are denoted as $\alpha = (\alpha_1, \dots, \alpha_{n_\alpha})$ and $\mathbf{b} = (b_1, \dots, b_{n_b})$, respectively.
- $\text{EncKey}(y, \alpha, \mathbf{b}) \rightarrow (m_1, m_2, \mathbf{k}(\mathbf{r}, \hat{\mathbf{r}}, \alpha, \mathbf{b}, y))$: On input $y \in \mathcal{Y}$, α and $\mathbf{b}$, this algorithm outputs a vector of polynomials $\mathbf{k} = (k_1, \dots, k_{m_3})$, with $m_3 \in \mathbb{N}$, defined over non-lone variables $\mathbf{r} = (r_1, \dots, r_{m_1})$ and lone variables $\hat{\mathbf{r}} = (\hat{r}_1, \dots, \hat{r}_{m_2})$. Specifically, the polynomial k_i is expressed as

$$k_i = \sum_{j \in [n_\alpha]} \delta_{i,j} \alpha_j + \sum_{j \in [m_2]} \hat{\delta}_{i,j} \hat{r}_j + \sum_{j \in [m_2], k \in [n_\alpha]} \hat{\delta}'_{1,i,j,k} \hat{r}_j \alpha_k \\ + \sum_{j \in [m_2], k \in [n_b]} \hat{\delta}'_{2,i,j,k} \hat{r}_j b_k + \sum_{j \in [m_1], k \in [n_b]} \delta_{i,j,k} r_j b_k,$$

for all $i \in [m_3]$, where $\delta_{i,j}, \hat{\delta}_{i,j}, \hat{\delta}'_{1,i,j,k}, \hat{\delta}'_{2,i,j,k}, \delta_{i,j,k} \in \mathbb{Z}_p$.

- $\text{EncCt}(x, \alpha, \mathbf{b}) \rightarrow (w_1, w_2, w'_2, c_M, \mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}, x), \mathbf{c}'(\mathbf{s}, \tilde{\mathbf{s}}, \alpha, x))$: On input $x \in \mathcal{X}$, α and $\mathbf{b}$, this algorithm outputs a blinding variable c_M and two vectors of polynomials $\mathbf{c} = (c_1, \dots, c_{w_3})$ and $\mathbf{c}' = (c'_1, \dots, c'_{w_4})$, with $w_3, w_4 \in \mathbb{N}$, defined over non-lone variables $\mathbf{s} = (s_1, s_2, \dots, s_{w_1})$, lone variables $\hat{\mathbf{s}} = (\hat{s}_1, \dots, \hat{s}_{w_2})$ and special lone variables $\tilde{\mathbf{s}} = (\tilde{s}_1, \dots, \tilde{s}_{w'_2})$. Specifically, the polynomial c_i is expressed as

$$c_i = \sum_{j \in [w_2]} \hat{\eta}_{i,j} \hat{s}_j + \sum_{j \in [w_2], k \in [n_b]} \hat{\eta}_{i,j,k} \hat{s}_j b_k + \sum_{j \in [w_1], k \in [n_b]} \eta_{i,j,k} s_j b_k,$$

for all $i \in [w_3]$, where $\hat{\eta}_{i,j}, \hat{\eta}_{i,j,k}, \eta_{i,j,k} \in \mathbb{Z}_p$, the polynomial c'_i is expressed as

$$c'_i = \sum_{j \in [n_\alpha], j' \in [w_1]} \eta'_{i,j,j'} \alpha_j s_{j'} + \sum_{j \in [w'_2]} \hat{\eta}'_{i,j} \tilde{s}_j + \sum_{j \in [w'_2], k \in [n_\alpha]} \hat{\eta}'_{i,j,k} \tilde{s}_j \alpha_k,$$

for all $i \in [w_4]$, where $\eta'_{i,j,j'}, \hat{\eta}'_{i,j}, \hat{\eta}'_{i,j,k} \in \mathbb{Z}_p$, and the variable c_M is expressed as

$$c_M = \sum_{j \in [w'_2]} \hat{\zeta}_j \tilde{s}_j + \sum_{j \in [w'_2], k \in [n_\alpha]} \hat{\zeta}_{j,k} \tilde{s}_j \alpha_k + \sum_{j \in [n_\alpha], j' \in [w_1]} \zeta_{j,j'} \alpha_j s_{j'},$$

where $\hat{\zeta}_j, \hat{\zeta}_{j,k}, \zeta_{j,j'} \in \mathbb{Z}_p$.

- $\text{Pair}(x, y) \rightarrow (\mathbf{e}, \mathbf{E}, \overline{\mathbf{E}})$: On input $x \in \mathcal{X}$, and $y \in \mathcal{Y}$, this algorithm outputs a vector $\mathbf{e} \in \mathbb{Z}_p^{w_4}$ and two matrices $\mathbf{E}$ and $\overline{\mathbf{E}}$ of sizes $w_1 \times m_3$ and $w_3 \times m_1$, respectively.

An approximate PES is correct for every $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ such that $P(x, y) = 1$, it holds that $\mathbf{e}\mathbf{c}'^\top + \mathbf{s}\mathbf{E}\mathbf{k}^\top + \mathbf{c}\overline{\mathbf{E}}\mathbf{r}^\top = c_M$.

Compiling an Approximate PES. Because the difference between PES and approximate PES is solely in the structure of the key and ciphertext polynomials, we can simply instantiate the Ven23 and ISABELLA compilers with an approximate PES. This is in part due to the level of abstraction on which those compilers are defined. For example, the ciphertext polynomials c_i are compiled into the ciphertext components $[c_i]_{\mathfrak{D}(c_i)}$. The compilers do not describe how this is computed (and essentially only state that they can be). Furthermore, approximate PES has the same types of variables that need to be sampled upon compilation.

B.2 Security Definition

We propose a statistical security definition for approximate PES that we use to generically prove security of the compiled scheme. To benefit from the results in ISABELLA, we prove security in relation to a compiled PES-Ven23 or PES-ISA. To this end, our statistical security notion considers the statistical distance between an approximate PES and a PES-Ven23 or PES-ISA. If this statistical distance is negligible, then the approximate PES is sufficiently close to the given PES-Ven23 or PES-ISA. We use this to reduce the security of the compiled approximate PES to the security of the compiled PES-Ven23 or PES-ISA.

Statistical Distance. We define the statistical difference between two distributions X and Y over the possible values V as $\Delta(X, Y) = \frac{1}{2} \sum_{v \in V} |\Pr[X = v] - \Pr[Y = v]|$. We call two distributions statistically indistinguishable if the statistical difference is negligible in the security parameter, i.e., $\Delta(X, Y) \leq \text{negl}(\lambda)$. (Note that this also means that X and Y are implicitly dependent on λ .)

Definition 15 (Statistically close approximate PES). Let $\tilde{\Gamma}_{\text{PES-Ven23}}$ be an approximate PES and $\Gamma_{\text{PES-Ven23}}$ be a PES-Ven23, where we denote the distinction between the key and ciphertext polynomials generated with EncKey and EncCt by writing $\tilde{\mathbf{k}}, \tilde{\mathbf{c}}$ and $\tilde{\mathbf{c}}'$ for the key and ciphertext polynomials of the approximate PES. Let p be a prime number for which we can find a suitable pairing group for security parameter λ . Consider the following distributions for $x \in \mathcal{X}$ and $y \in \mathcal{Y}$:

- (i) $X_{x,y} = \{(\alpha, \mathbf{b}, \mathbf{r}, \mathbf{k}(\mathbf{r}, \hat{\mathbf{r}}, \alpha, \mathbf{b}, y), \mathbf{s}, c_M, \mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}, x), \mathbf{c}'(\mathbf{s}, \tilde{\mathbf{s}}, \alpha, x)) \mid \alpha_k, b_k, r_j, \hat{r}_j, s_j, \hat{s}_j, \tilde{s}_j \in_R \mathbb{Z}_p\}_{x,y}$ (i.e., the distribution of the regular PES)
- (ii) $Y_{x,y} = \{(\alpha, \mathbf{b}, \mathbf{r}, \tilde{\mathbf{k}}(\mathbf{r}, \hat{\mathbf{r}}, \alpha, \mathbf{b}, y), \mathbf{s}, \tilde{c}_M, \tilde{\mathbf{c}}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}, x), \tilde{\mathbf{c}}'(\mathbf{s}, \tilde{\mathbf{s}}, \alpha, x)) \mid \alpha_k, b_k, r_j, \hat{r}_j, s_j, \hat{s}_j, \tilde{s}_j \in_R \mathbb{Z}_p\}_{x,y}$ (i.e., the distribution of the approximate PES)

If for all x, y the statistical difference $\Delta(X_{x,y}, Y_{x,y}) \leq \text{negl}(\lambda)$, i.e., the two distributions are statistically indistinguishable, then we say that $\tilde{\Gamma}_{\text{PES-Ven23}}$ is statistically, or sufficiently, close to $\Gamma_{\text{PES-Ven23}}$.

B.3 Security of Compiled Approximate PES

We use our new security definition to prove security of compiled approximate PESs.

Theorem 1. *Let $\Gamma_{\text{PES-Ven23}}$ be a PES-Ven23 for which the special selective symbolic property holds, and let $\tilde{\Gamma}_{\text{PES-Ven23}}$ be an approximate PES that is statistically close to $\Gamma_{\text{PES-Ven23}}$. Then compiling the approximate PES with the a compatible group distribution, FDH assignment and split-predicate mold yields a scheme with the same security properties as the compiled version of $\Gamma_{\text{PES-Ven23}}$.*

Proof. Let $\tilde{\Gamma}$ be the compiled scheme obtained from $\tilde{\Gamma}_{\text{PES-Ven23}}$ and Γ from $\Gamma_{\text{PES-Ven23}}$. We show that the security games for $\tilde{\Gamma}$ and Γ are statistically indistinguishable. This follows from the fact that the keys and ciphertexts associated with the polynomials are statistically indistinguishable. That is, when considering the secret keys in the key query phases, which are of the form

$$\text{USK}_S = (\mathcal{S}, \{[r_j]_{\mathfrak{D}(r_j)} \mid r_j \in \mathbf{r} \wedge \mathcal{F}_1(r_j) = 0\}, \{[k_i]_{\mathfrak{D}(k_i)}\}_{k_i \in \mathbf{k}}),$$

it follows from the fact that $\tilde{\Gamma}$ and Γ are statistically close that the secret keys are indistinguishable. Similarly, the challenge ciphertexts are statistically indistinguishable. Therefore, the security games are indistinguishable. $\square$

B.4 Approximate PES with More Efficient Access Structures

We define a transformation for PESs with the less practical secret sharing mechanism, and prove its security generically by using our notion of statistically close approximate PES.

Lemma 1. *Let λ be the security parameter, let Γ be a PES with ciphertext polynomials containing access structures of the form $\lambda_j = A_{j,1}sb + \sum_{k \in [n_2]} A_{j,k}v_k$, and let $p \geq 2^\lambda$ be a prime number. That is, let $\mathcal{I} \subseteq [w_3]$ be a subset of ciphertext polynomial indices such that there exists a bijection $f: [n_2] \rightarrow \mathcal{I}$, i.e., $c_{f(j)}$ contains monomials λ_j , and v_k appear nowhere else. Let $\tilde{\Gamma}$ be the approximate PES that we construct by replacing λ_j in each $c_{f(j)}$ by $\tilde{\lambda}_j = A_{j,1}sb + \sum_{k \in [n_2]} A_{j,k}\tilde{v}_kb$. Then $\tilde{\Gamma}$ is statistically close to Γ .*

Proof. We show that the distributions of the regular PES and the approximate PES are statistically indistinguishable. Let $x \in \mathcal{X}$ and $y \in \mathcal{Y}$. Because the distributions of the non-lone variables $\mathbf{r}, \mathbf{s}$ and the polynomials $\mathbf{k}, c_M, \mathbf{c}'$ are the same in the two PESs, their distributions are the same. For ciphertext polynomials c_i with $i \notin \mathcal{I}$, this is also the case.

For $i \in \mathcal{I}$, the regular PES has the polynomials c_i with $\lambda_{f^{-1}(i)}$ and the approximate PES has polynomials $\tilde{c}_i$ with $\tilde{\lambda}_{f^{-1}(i)}$. To show that these distributions are indistinguishable, we have to show that the distributions of $\lambda_{f^{-1}(i)}$ and $\tilde{\lambda}_{f^{-1}(i)}$ are statistically close. This is because of the fact that v_k do not occur in other polynomials, and that, with probability $\frac{p-1}{p}$, we have that $b \neq 0$. That is, if $b \neq 0$, then we have that, for each $v_k \in \mathbb{Z}_p$, there exists exactly one $\tilde{v}_k \in \mathbb{Z}_p$ such that $v_k = \tilde{v}_kb \bmod p$ and vice versa. Hence, the distributions of v_k and $\tilde{v}_kb$ are indistinguishable if $b \neq 0$. (If $b = 0$, then the two are distinguishable.) By extension, λ_j and $\tilde{\lambda}_j$ are statistically indistinguishable.

In sum, the statistical distance between $X_{x,y}$ and $Y_{x,y}$ (which are the distributions for Γ and $\tilde{\Gamma}$, respectively) is $\frac{1}{p}$ (which is the probability that $b = 0$). Because $\frac{1}{p} \leq \frac{1}{2^\lambda} = \text{negl}(\lambda)$, the approximate PES $\tilde{\Gamma}$ is statistically close to the PES Γ . $\square$

C Type System: Extended Motivation

The type system makes our compiler more flexible, usable and reliable. It allows the detection and prevention of subtle problems, which we mention in the text. Below, we give elaborate examples to fully demonstrate the usefulness of our formal model in some non-trivial cases.

Flexibility. In this example, we illustrate that the tracking of types is useful to automatically generate code in statically typed languages without type inference. This greatly increases the flexibility of how our tool can be used. Consider the cipher polynomial

$$c_{\text{att}} : G = s \cdot b_{\text{att.toAuth}} \quad \forall \text{att} \in \text{LSSSRowToAttr}(\text{LSSSRows}).$$

After code generation, we obtain a for-loop as shown in Listing 2. In this example, the for-loop iterates over all LSSS rows. Each LSSS row is mapped to its corresponding attribute before being used in the body of the loop. Then, the attribute is further mapped to an authority, since $b_{\text{att.toAuth}}$ is indexed by the corresponding authority.

```

1  for (XXX att_global : get_lsss_rows()) {
2      YYY att = ls_row_to_attr(att_global)
3      ...
4      // use att here to build index for c
5      ...
6      ZZZ att_local = attr_to_auth(att)
7      // use att_local here to build index for b
8      ...
9  }
```

Listing 2: Example code with placeholders for type annotations in an statically typed language (C-like syntax).

The placeholders **XXX**, **YYY** and **ZZZ** represent the types of LSSS rows, attributes and authorities in the backend. Note that we make no assumptions about these types. Every backend can use any type here (and specify so in our export layer). Even more so, some backends might require type annotations, and by including the types we can generate code for statically typed languages without type inference. Our type system also enables us to emit the correct function call to `attr_to_auth` without relying on overloading functionality of the backend technologies. In this way, the formal model gives us substantially more flexibility regarding the backends we can support.

Usability. This examples demonstrates that our type system allows the detection of dangerous variable naming that could potentially lead to insecurities when implemented (if no rigorous restrictions are posed on either the input format or backend design - which impede usability). Consider the key polynomials

$$\mathbf{k}_{1,1} : \mathbb{G} = \mathbf{a}_1 + \mathbf{r} \cdot \mathbf{b}_1 \quad \forall \mathbf{l} \in \text{Auths} \quad \text{and} \quad \mathbf{k}_1 : \mathbb{G} = \mathbf{r} \cdot \mathbf{b}_1 \quad \forall \mathbf{l} \in \text{UserAttrs}.$$

The crucial point here are the common variables $\mathbf{b}_1$. Notice that the $\mathbf{b}_1$ in both polynomials are *syntactically identical*, but refer to different variables. The way a human cryptographer sees this is by “tracking the types” mentally. For instance, the first polynomial is quantified over all authorities, whereas the second is quantified all attributes. Since these sets are (implicitly) assumed to be disjoint, the indices $\mathbf{l}$ will never collide and hence the $\mathbf{b}_1$ are always distinct. In an implementation, however, attributes and authorities could be represented as strings, allowing for collisions during lookup in the MSK. In the worst case, this results in a correct (i.e., no backend tests fail) scheme which is insecure (e.g., collisions can be exploited) and goes unnoticed. Based on our formal model, the compiler detects the common variables $\mathbf{b}_1$ as *similar* (Definition 12) and reports this ambiguity to the user.

Instead of tracking the types, this could also be resolved by placing stronger assumptions on the backend (i.e., must guarantee disjointness) or the input format (e.g., variables must be renamed). This would make **pracy** more error-prone to use and less user-friendly.

Reliability. This examples shows how we automatically verify the correctness of the FDH assignment (which needs to be compatible with the group assignment) based on our notion of equivalence (Definition 11). Without our type system, we would not be able to verify this correctness automatically without requiring more user input or risking dysfunctional (or even insecure) code. Consider the key polynomials

$$\mathbf{k}_{1,1} : \mathbb{G} = \mathbf{a}_1 + \mathbf{r} \cdot \mathbf{b}_1 \quad \forall \mathbf{l} \in \text{Auths} \quad \text{and} \quad \mathbf{k}_{2,j} : \mathbb{H} = \mathbf{r}_{\text{GID}} + \mathbf{r} \cdot \mathbf{b}_{j.\text{toAuth}} \quad \forall \mathbf{j} \in \text{UserAttrs}.$$

In this case the common variables $\mathbf{b}_1$ and $\mathbf{b}_{j.\text{auth}}$ are *syntactically different*, but they refer to the same common variables. Our compiler infers this by tracking the types of the indices. Moreover, **pracy** can deduce that $\mathbf{b}_1$ cannot be generated via an FDH, because that would contradict the group assignment of the key polynomials. Hence, specifying $\mathbf{b}_1$ in the FDH assignment would be incorrect. Note that this issue can only be identified by jointly analyzing both key polynomials, their group assignments, and the FDH map.

Not having a type system would mean that we cannot verify this correctness, because we are not able to tell that the two occurrences of the common variable are the same. Hence, we have to simply assume FDH correctness and the implement the scheme. This is undesirable, because the incorrectness might go unnoticed. It could work in the case that we use symmetric pairings (which is supported by, e.g., Charm), because then $\mathbb{G} = \mathbb{H}$, but

does not work in the asymmetric setting (which we require for practice), because hashing in $\mathbb{G}$ and hashing in $\mathbb{H}$ yield a different “exponent” even when queried on the same input. Hence, in the latter case, the backend could detect this incorrectness, *if* we have sufficient decryption test cases. However, this is not generally the case, and it is also not clear what kind of incorrectness issue the backend raises. In contrast, our type system always catches these issues and reports that the group assignment is inconsistent with the FDH assignment.

D IR Reference

The IR is the language we use internally to represent the generated code. It is vital to keep `pracy` independent of any backends and maintain a separation of concern. At the same time, the set of IR statements defines precisely the API we expect each backend to implement. The table below lists all statements along with a short semantic description.

Statement	Description
<code>Comment(text)</code>	Add the text <code>text</code> as a comment.
<code>Loop(var, type, set, body)</code>	Execute <code>body</code> once for each element <code>var</code> of type <code>type</code> in <code>set</code> .
<code>Store(trgt, src)</code>	Store the value of <code>src</code> in <code>trgt</code> .
<code>StoreExpr(trgt, expr)</code>	Store the result of evaluating <code>expr</code> in <code>trgt</code> .
<code>Alloc(trgt, type, expr)</code>	Same as <code>StoreExpr</code> but declare <code>trgt</code> as a new variable of type <code>type</code> .
<code>ResetZ(trgt)</code>	Set <code>trgt</code> to 0 (in $\mathbb{Z}_p$).
<code>ResetG(trgt)</code>	Set <code>trgt</code> to the identity element regarding <code>AddG</code> .
<code>ResetH(trgt)</code>	Set <code>trgt</code> to the identity element regarding <code>AddH</code> .
<code>ResetGt(trgt)</code>	Set <code>trgt</code> to the identity element regarding <code>AddGt</code> .
<code>SampleZ(trgt)</code>	Set <code>trgt</code> to a randomly sampled element from $\mathbb{Z}_p$.
<code>AddZ(trgt, lhs, rhs)</code>	Set <code>trgt</code> to the sum of <code>lhs</code> and <code>rhs</code> (in $\mathbb{Z}_p$).
<code>MulZ(trgt, lhs, rhs)</code>	Set <code>trgt</code> to the product of <code>lhs</code> and <code>rhs</code> (in $\mathbb{Z}_p$).
<code>SetZ(trgt, val)</code>	Set <code>trgt</code> to the value <code>val</code> (in $\mathbb{Z}_p$).
<code>NegZ(trgt, src)</code>	Set <code>trgt</code> to the additive inverse of <code>src</code> (in $\mathbb{Z}_p$).
<code>InvZ(trgt, src)</code>	Set <code>trgt</code> to the multiplicative inverse of <code>src</code> (in $\mathbb{Z}_p$).
<code>LiftG(trgt, src)</code>	Set <code>trgt</code> to $[\text{src}]_{\mathbb{G}}$.
<code>AddG(trgt, lhs, rhs)</code>	Set <code>trgt</code> to $\text{lhs} \circ_{\mathbb{G}} \text{rhs}$.
<code>ScaleG(trgt, coeff, src)</code>	Set <code>trgt</code> to $\text{coeff} \cdot \text{src}$ (alternatively: $\text{src}^{\text{coeff}}$) (in $\mathbb{G}$).
<code>FdhG(trgt, idx, arg)</code>	Set <code>trgt</code> to the result of $\mathcal{F}_{\text{idx}}(\text{arg})$ (in $\mathbb{G}$).
<code>LiftH(trgt, src)</code>	Set <code>trgt</code> to $[\text{src}]_{\mathbb{H}}$.
<code>AddH(trgt, lhs, rhs)</code>	Set <code>trgt</code> to $\text{lhs} \circ_{\mathbb{H}} \text{rhs}$.
<code>ScaleH(trgt, coeff, src)</code>	Set <code>trgt</code> to $\text{coeff} \cdot \text{src}$ (alternatively: $\text{src}^{\text{coeff}}$) (in $\mathbb{H}$).
<code>FdhH(trgt, idx, arg)</code>	Set <code>trgt</code> to the result of $\mathcal{F}_{\text{idx}}(\text{arg})$ (in $\mathbb{H}$).
<code>LiftGt(trgt, src)</code>	Set <code>trgt</code> to $[\text{src}]_{\mathbb{G}_T}$.
<code>AddGt(trgt, lhs, rhs)</code>	Set <code>trgt</code> to $\text{lhs} \circ_{\mathbb{G}_T} \text{rhs}$.
<code>ScaleGt(trgt, coeff, src)</code>	Set <code>trgt</code> to $\text{coeff} \cdot \text{src}$ (alternatively: $\text{src}^{\text{coeff}}$) (in $\mathbb{G}_T$).
<code>InvGt(trgt, src)</code>	Set <code>trgt</code> to the inverse (regarding $\circ_{\mathbb{G}_T}$) of <code>src</code> (in $\mathbb{G}_T$).
<code>Pair(trgt, src_g, src_h)</code>	Set <code>trgt</code> to $e(\text{src}_g, \text{src}_h)$.
<code>GetRgidG(trgt)</code>	Set <code>trgt</code> to the current r_{GID} (in $\mathbb{G}$).
<code>GetRgidH(trgt)</code>	Set <code>trgt</code> to the current r_{GID} (in $\mathbb{H}$).
<code>GetMu(trgt, j)</code>	Set <code>trgt</code> to μ_j , i.e., the j -th share of 0.
<code>GetLambda(trgt, j)</code>	Set <code>trgt</code> to λ_j , i.e., the j -th share of the secret s .

Statement	Description
GetEpsilon(<i>trgt</i> , <i>j</i>)	Set <i>trgt</i> to ε_j , i.e., the coefficient for the <i>j</i> -th LSSS row.
GetXAttr(<i>trgt</i> , <i>attr</i>)	Set <i>trgt</i> to the numeric representation x_{attr} of <i>attr</i> .
GetXAttrAlt(<i>trgt</i> , <i>j</i>)	Set <i>trgt</i> to the numeric representation of the <i>alternative</i> attribute for the <i>j</i> -th LSSS row (OT-type negations).
GetSecret(<i>trgt</i>)	Set <i>trgt</i> to the master secret <i>s</i> .
SetIndex(<i>lit</i>)	Set the index string to <i>lit</i> .
AppendIndexLiteral(<i>lit</i>)	Append the literal <i>lit</i> to the index string.
AppendIndex(<i>src</i> , <i>conv</i>)	Append the result of converting <i>src</i> to a string using <i>conv</i> to the index string.

E Scheme Overview

For our evaluation, we take five schemes from literature as well as variations thereof and run them through our compiler. The Table below describes the schemes and variations in detail. For each scheme we give a literature reference to the scheme itself and/or the generalized pair-encoding scheme (GPES), depending on what is available. Note that some variations are “not possible”, i.e., we cannot remove the deduplication mapping τ if the scheme at hand does not use it already. In general, schemes ending in `_od`, `_oe` or `_ok` correspond to variants optimized for decryption, encryption and key generation as presented in ABE Squared [dlPVA22a]. Those ending in `_xx` are “weird” variants, which serve testing purposes only. We do not of any practical use-cases.

Scheme	Description	Reference
a_0_oe	Optimized encryption	Construction 4.5 [Ven23a]
a_0_ok	Optimized key generation	Construction 4.5 [Ven23a]
a_1_xx	No FDHs	$\simeq$ Construction 4.5 [Ven23a]
a_2_xx	No FDHs + other group assignment	$\simeq$ Construction 4.5 [Ven23a]
a_3_xx	Hashed $\{r_l\}_{l \in [n_{\text{aut}}]}$	$\simeq$ Construction 4.5 [Ven23a]
a_4_xx	Hashed $\{b'_l\}_{l \in [n_{\text{aut}}]}$	$\simeq$ Construction 4.5 [Ven23a]
a_5_xx	No deduplication mapping τ	$\simeq$ Construction 4.5 [Ven23a]
a_6_xx	Single authority	$\simeq$ Construction 4.5 [Ven23a]
a_7_xx	Scaling factor	$\simeq$ Construction 4.5 [Ven23a]
b_0_oe	Optimized encryption	Definition 16 [Ven23b]
b_0_ok	Optimized key generation	Definition 16 [Ven23b]
b_1_xx	No FDHs	$\simeq$ Definition 16 [Ven23b]
b_2_xx	No FDHs + other group assignment	$\simeq$ Definition 16 [Ven23b]
b_3_xx	Hashed $\{r_l\}_{l \in [n_{\text{aut}}]}$	$\simeq$ Definition 16 [Ven23b]
b_4_xx	Hashed $\{b'_l\}_{l \in [n_{\text{aut}}]}$	$\simeq$ Definition 16 [Ven23b]
b_5_xx	No deduplication mapping τ	$\simeq$ Definition 16 [Ven23b]
b_6_xx	Single authority	$\simeq$ Definition 16 [Ven23b]
b_7_xx	Scaling factor	$\simeq$ Definition 16 [Ven23b]
c_0_oe	Optimized encryption	Wat11-I [BSW07, Ven23a]
c_0_ok	Optimized key generation	Wat11-I [BSW07, Ven23a]
c_1_oe	Optimized encryption	Wat11-IV [BSW07, Ven23a]
c_1_ok	Optimized key generation	Wat11-IV [BSW07, Ven23a]
c_2_xx	Other group assignment	$\simeq$ Wat11-IV [BSW07, Ven23a]
c_3_xx	Hashed <i>r</i>	$\simeq$ Wat11-I [BSW07, Ven23a]
c_4_xx	Hashed <i>b</i>	$\simeq$ Wat11-I [BSW07, Ven23a]

Scheme	Description	Reference
c_5_xx	–	–
c_6_xx	–	–
c_7_xx	Scaling factor	$\simeq$ Wat11-I [BSW07, Ven23a]
d_0_oe	Optimized encryption	RW13 [RW13, Ven23a]
d_o_ok	Optimized key generation	RW13 [RW13, Ven23a]
d_1_xx	Other group assignment	$\simeq$ RW13 [RW13, Ven23a]
d_2_xx	Hashed r and $\{r_{\text{attr}}\}_{\text{attr} \in \mathcal{S}}$	$\simeq$ RW13 [RW13, Ven23a]
d_3_xx	Hashed b, b', b_0 and b_1	$\simeq$ RW13 [RW13, Ven23a]
d_4_xx	Hashed b, b' and $\{r_{\text{attr}}\}_{\text{attr} \in \mathcal{S}}$	$\simeq$ RW13 [RW13, Ven23a]
d_5_xx	–	–
d_6_xx	–	–
d_7_xx	Scaling factor	$\simeq$ RW13 [RW13, Ven23a]
e_0_od	Optimized decryption	AC17 [AC17, Ven23a]
e_0_oe	Optimized encryption	AC17 [AC17, Ven23a]
e_0_ok	Optimized key generation	AC17 [AC17, Ven23a]
e_1_od	Optimized decryption	AC17-LU [AC17, Ven23a]
e_1_oe	Optimized encryption	AC17-LU [AC17, Ven23a]
e_1_ok	Optimized key generation	AC17-LU [AC17, Ven23a]
e_2_xx	Other group assignment	$\simeq$ AC17 [AC17, Ven23a]
e_3_xx	Hashed r	$\simeq$ AC17 [AC17, Ven23a]
e_4_xx	Hashed b	$\simeq$ AC17 [AC17, Ven23a]
e_5_xx	No deduplication mapping τ	$\simeq$ AC17 [AC17, Ven23a]
e_6_xx	–	–
e_7_xx	Scaling factor	$\simeq$ AC17 [AC17, Ven23a]